VITE

CHEAT SHEETS

A Complete Reference for Vite 8 — Rolldown & Oxc Era

Dev Server · HMR · Config · Env & Modes · Assets · Glob Imports · CSS & Tailwind · Rolldown Builds · Library Mode · Plugins · SSR · Environment API · Vitest · Vite 8 Migration

39 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & How Vite Works
02 Creating a Project
03 Project Structure & index.html
04 CLI Commands

CONFIGURATION

05 Config File Basics
06 Shared Options
07 Resolve & Path Aliases
08 Env Variables & Modes
09 TypeScript in Vite

DEV SERVER

10 Dev Server Options
11 Proxy, CORS & Headers
12 Hot Module Replacement
13 Dependency Pre-Bundling

ASSETS & IMPORTS

14 Static Assets
15 Glob Imports
16 JSON, WebAssembly & Web Workers
17 Dynamic Imports & Preloading

CSS & STYLING

18 CSS & CSS Modules
19 Preprocessors, PostCSS & Lightning CSS
20 Tailwind & CSS Build Options

BUILDING FOR PRODUCTION

21 Build Basics
22 Rolldown Options & Chunking
23 Minification & Bundle Size
24 Library Mode
25 Preview & Static Deployment
26 Browser Support & Legacy Builds

PLUGINS

27 Using Plugins
28 Framework Plugins
29 Writing Plugins: Core Hooks
30 Writing Plugins: Server & HTML Hooks

SSR & ADVANCED

31 Server-Side Rendering
32 Environment API
33 JavaScript API
34 Performance & Bundled Dev Mode

TESTING, DEBUGGING & MIGRATION

35 Testing with Vitest
36 Debugging & Troubleshooting
37 Migrating to Vite 8

QUICK REFERENCE

38 Quick Reference: CLI & Config
39 Quick Reference: Imports, Env & Plugin API

Vite Reference

Table of Contents

39 reference sheets · syntax · examples · notes

Sheet 1
Introduction & How Vite Works
native ESM dev server · Rolldown · Oxc · dev vs build · dependency pre-bundling · Vite 8

Sheet 2
Creating a Project
npm create vite@latest · --template react-ts · templates · package.json scripts · Node 20.19+ / 22.12+

Sheet 3
Project Structure & index.html
index.html as entry · <script type="module"> · public/ · src/ · root · vite-env.d.ts

Sheet 4
CLI Commands
vite · vite build · vite preview · --host · --port · --mode · --force · --config

Sheet 5
Config File Basics
vite.config.ts · defineConfig · ({ command, mode }) · async config · mergeConfig · --configLoader

Sheet 6
Shared Options
root · base · mode · define · publicDir · cacheDir · logLevel · clearScreen

Sheet 7
Resolve & Path Aliases
resolve.alias · resolve.tsconfigPaths · extensions · conditions · dedupe · mainFields

Sheet 8
Env Variables & Modes
import.meta.env · VITE_ prefix · .env.[mode].local · loadEnv · envPrefix · MODE / DEV / PROD / BASE_URL

Sheet 9
TypeScript in Vite
transpile only · tsc --noEmit · isolatedModules · vite/client types · ImportMetaEnv · oxc options

Sheet 10
Dev Server Options
server.host · port · strictPort · open · https · allowedHosts · forwardConsole

Sheet 11
Proxy, CORS & Headers
server.proxy · changeOrigin · rewrite · ws · server.cors · server.headers

Sheet 12
Hot Module Replacement
import.meta.hot · accept() · dispose() · hot.data · invalidate() · server.hmr · full reload

Sheet 13
Dependency Pre-Bundling
optimizeDeps.include · exclude · optimizeDeps.rolldownOptions · --force · node_modules/.vite · linked packages

Sheet 14
Static Assets
import url · ?url · ?raw · ?inline · public/ · assetsInlineLimit · new URL(path, import.meta.url)

Sheet 15
Glob Imports
import.meta.glob · eager · import: 'default' · query · caseSensitive · !negative patterns

Sheet 16
JSON, WebAssembly & Web Workers
named JSON imports · .wasm?init · Wasm ESM import · ?worker · new Worker(new URL()) · worker.format

Sheet 17
Dynamic Imports & Preloading
import() · variables in import() · /* @vite-ignore */ · modulepreload · build.modulePreload · vite:preloadError

Sheet 18
CSS & CSS Modules
import './x.css' · *.module.css · ?inline · @import inlining · url() rebasing · css.modules

Sheet 19
Preprocessors, PostCSS & Lightning CSS
sass-embedded · less · postcss.config.js · css.transformer: 'lightningcss' · css.preprocessorOptions · css.devSourcemap

Sheet 20
Tailwind & CSS Build Options
@tailwindcss/vite · @import "tailwindcss" · build.cssCodeSplit · build.cssMinify · cssTarget

Sheet 21
Build Basics
vite build · outDir · assetsDir · emptyOutDir · build.sourcemap · build.manifest · build.target

Sheet 22
Rolldown Options & Chunking
build.rolldownOptions · input (multi-page) · output.codeSplitting · chunkFileNames · external · manualChunks (deprecated)

Sheet 23
Minification & Bundle Size
build.minify (Oxc) · minify.compress.drop · treeshake · chunkSizeWarningLimit · bundle analysis · build.chunkImportMap

Sheet 24
Library Mode
build.lib · entry · formats · fileName · external · peerDependencies · package.json exports

Sheet 25
Preview & Static Deployment
vite preview · preview.port · base for subpaths · SPA fallback · GitHub Pages · Netlify / Vercel · cache headers

Sheet 26
Browser Support & Legacy Builds
'baseline-widely-available' · build.target · @vitejs/plugin-legacy · modulePreload polyfill · no ES5 output

Sheet 27
Using Plugins
plugins array · enforce: 'pre' / 'post' · apply: 'build' · conditional plugins · Rollup plugin compatibility · plugin order

Sheet 01

Introduction & How Vite Works

native ESM dev server · Rolldown · Oxc · dev vs build · dependency pre-bundling · Vite 8

WHAT VITE IS

Vite (French for "quick", pronounced *veet*) is a front-end build tool created by Evan You in 2020. In development it serves your source over **native ES modules** and transforms each file only when the browser asks for it, so startup time doesn't grow with app size. For production, `vite build` bundles with **Rolldown** (Rust) and transforms and minifies with **Oxc**. Vite 8 (March 2026) replaced the old esbuild + Rollup pair with this single toolchain.

NOTES

- Requires Node.js `20.19+` or `22.12+`; Vite 8 ships as ESM only
- Framework-agnostic: React, Vue, Svelte, Solid, Preact, Lit, vanilla
- Powers Vitest, Nuxt, SvelteKit, Astro, React Router, Analog
- Default ports: dev `5173`, preview `4173`

FROM ZERO TO RUNNING APP

```
npm create vite@latest hello -- --template vanilla-ts
cd hello
npm install
npm run dev        # ➜ Local: http://localhost:5173/
npm run build      # → dist/index.html + dist/assets/*-[hash].js
npm run preview    # serves dist/ at http://localhost:4173/
```

CODE EXPLAINED

- → `create vite` — scaffolds a project from a template
- → `npm run dev` — runs `vite`; no bundling, instant start
- → `npm run build` — runs `vite build`; output goes to `dist/`
- → `npm run preview` — serves the built output locally to check it
- → Edit a file while dev runs → **HMR** updates the page in place

DEV SERVER VS BUILD

ASPECT	VITE (DEV)	VITE BUILD
Modules	Native ESM, one request per file	Bundled, hashed chunks
Transform	Oxc, on demand per request	Oxc + Rolldown, whole graph
Dependencies	Pre-bundled once to `.vite/`	Bundled with your code
Updates	HMR over WebSocket	Rebuild (`--watch`)
Output	Served from memory	Files in `dist/`

VITE 8 TOOLCHAIN

PIECE	JOB
Rolldown	Bundler for build and dep pre-bundling
Oxc	Parse, TS/JSX transform, JS minify
Lightning CSS	CSS minify (default in build)
Plugin API	Rollup-style hooks + Vite-only hooks
esbuild	No longer a dependency; opt-in only

⚠ COMMON MISTAKES

- Opening `index.html` straight from disk — it needs the dev server or a built `dist/` served over HTTP
- Expecting `vite` to type-check — it strips types only; run `tsc --noEmit` separately
- Deploying the project root instead of `dist/` — only the build output is production-ready
- Following pre-2026 tutorials that configure `esbuild` or `rollupOptions` without knowing they're renamed

✓ TIPS & BEST PRACTICES

- Check your version with `npx vite --version` before copying config from docs
- Use `npm create vite@latest` so you always get the current templates
- Run `vite preview` before deploying to catch base-path and asset issues
- Read sheet 37 when upgrading a Vite 7 project

QUICK SUMMARY

- `vite` — fast dev server on native ESM + HMR
- `vite build` — Rolldown bundle into `dist/`
- `vite preview` — serve `dist/` locally
- `Oxc` — transforms TS/JSX and minifies JS

Creating a Project

SCAFFOLDING

`create-vite` scaffolds a ready-to-run project: an `index.html` entry, a `src/` folder, a config file and the three npm scripts. Run it without arguments for interactive prompts, or pass a project name and `--template` to skip them. The same command works through npm, pnpm, yarn, bun and deno.

NOTES

- Use `.` as the name to scaffold into the current folder
- npm 7+ needs the extra `--` before `--template`
- Templates ending in `-ts` add TypeScript and `tsconfig` files
- Community templates: `npx degit user/repo my-app`

CREATE COMMANDS

```
npm create vite@latest                                  # interactive prompts
npm create vite@latest my-app -- --template react-ts
pnpm create vite my-app --template vue-ts
yarn create vite my-app --template svelte-ts
bun create vite my-app --template solid-ts
deno init --npm vite my-app --template preact-ts
npm create vite@latest . -- --template vanilla          # current folder

# Add Vite to an existing project
npm install -D vite
npx vite                                                # serves ./index.html
```

CODE EXPLAINED

- → `@latest` — avoids a stale cached version of `create-vite`
- → `--template` — skips the framework and variant prompts
- → `pnpm` / `yarn` / `bun` — no `--` separator needed
- → `npm install -D vite` — Vite is always a dev dependency
- → Bare `npx vite` works with no config file at all

OFFICIAL TEMPLATES

TEMPLATE	FRAMEWORK
`vanilla` / `vanilla-ts`	Plain JS / TS
`react` / `react-ts`	React + @vitejs/plugin-react
`vue` / `vue-ts`	Vue 3 + @vitejs/plugin-vue
`svelte` / `svelte-ts`	Svelte 5
`solid` / `solid-ts`	SolidJS
`preact` / `preact-ts`	Preact
`lit` / `lit-ts`	Lit web components

GENERATED PACKAGE.JSON

KEY	VALUE
`"type"`	"module" — config and scripts are ESM
`"dev"`	vite
`"build"`	vite build (TS: tsc -b && vite build)
`"preview"`	vite preview
`devDependencies`	vite, plugin, typescript

⚠ COMMON MISTAKES

- Running `npm create vite my-app --template react` without `--` on npm 7+ — the flag is swallowed and you get prompts
- Installing `vite` as a regular dependency — it's build tooling and belongs in `devDependencies`
- Using an old Node version — Vite 8 refuses to start below 20.19 / 22.12
- Scaffolding into a non-empty folder and letting it overwrite files

✓ TIPS & BEST PRACTICES

- Keep the TS template's `tsc -b && vite build` so CI fails on type errors
- Pin Node with `"engines"` or `.nvmrc` so teammates match
- Prefer framework starters (Nuxt, SvelteKit, React Router) when you need routing and SSR
- Commit the lockfile right after scaffolding

QUICK SUMMARY

- `npm create vite@latest` — interactive scaffold
- `--template x-ts` — pick framework + TypeScript
- `npm i -D vite` — add to an existing app
- `dev` / `build` / `preview` — the three scripts

Project Structure & index.html

THE HTML ENTRY

In Vite, **`index.html` is the entry point**, not an afterthought in `public/`. It sits at the project root and loads your code with `<script type="module" src="/src/main.ts">`. Vite parses the HTML, follows that script and every import from it, and rewrites URLs at build time. Files in `public/` are copied as-is and served from `/`.

NOTES

- `root` defaults to the folder with `vite.config.*` / `process.cwd()`
- Absolute URLs in HTML (`/src/...`) resolve from `root`
- Each extra `.html` file can be another entry (multi-page app)
- `%VITE_TITLE%` placeholders in HTML are replaced from env vars

TYPICAL LAYOUT

```
my-app/
├── index.html          # entry, at root
├── public/             # copied as-is
│   └── favicon.svg     # → /favicon.svg
├── src/
│   ├── main.ts         # loaded by index.html
│   ├── App.tsx
│   ├── assets/logo.png # → hashed URL
│   ├── styles.css
│   └── vite-env.d.ts   # vite/client types
├── .env                # VITE_* variables
├── vite.config.ts
├── tsconfig.json
└── package.json
```

INDEX.HTML

```
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" href="/favicon.svg" />
    <meta name="viewport"
          content="width=device-width, initial-scale=1.0" />
    <title>%VITE_APP_TITLE%</title>
  </head>
  <body>
    <div id="app"></div>
    <script type="module" src="/src/main.ts"></script>
  </body>
</html>
```

WHERE FILES GO

LOCATION	USE FOR	URL IN BUILD
`src/assets/`	Images, fonts you import	/assets/logo-[hash].png
`public/`	robots.txt, favicon, files needing a fixed name	/robots.txt
`index.html`	Entry; links CSS and module scripts	/index.html
`src/`	All code Vite transforms	bundled into /assets/*.js

HTML HANDLING

IN INDEX.HTML	WHAT VITE DOES
`<script type="module" src>`	Entry; bundled and hashed
`<link rel="stylesheet" href>`	Processed CSS, hashed
`<img src="./x.png">`	Asset URL rewritten
`%VITE_X%`	Replaced with env value
`Inline <script type="module">`	Bundled like a file

⚠ COMMON MISTAKES

- Putting `index.html` inside `public/` — Vite won't treat it as the entry
- Leaving off `type="module"` — the script isn't processed and imports fail
- Importing from `public/` (`import x from '/public/x.png'`) — reference public files by URL only
- Changing `root` without updating `publicDir` and `envDir`, which resolve relative to it

✓ TIPS & BEST PRACTICES

- Keep `src/vite-env.d.ts` so `import.meta.env` and asset imports are typed
- Use `public/` only for files that must keep their exact name
- Set `root: 'src'` only if you want `index.html` inside `src/`
- Add more `.html` entries via `build.rolldownOptions.input` (sheet 22)

QUICK SUMMARY

- `index.html` — root-level entry, not in public/
- `<script type="module">` — where Vite starts
- `public/` — static, unhashed, served at /
- `src/assets` — imported, hashed, optimized

CLI Commands

COMMANDS

The `vite` binary has three main commands: `vite` (alias `vite dev` / `vite serve`) starts the dev server, `vite build` creates the production bundle, and `vite preview` serves that bundle. CLI flags override the matching config options, and an optional `[root]` argument changes the project root. While the dev server runs, single-key shortcuts (h for help) control it from the terminal.

NOTES

- Run through npm scripts or `npx vite …`
- Flags beat `vite.config.*` values
- `--mode` changes which `.env.[mode]` files load
- `vite --help` / `vite build --help` list every flag

EVERYDAY COMMANDS

```
vite                          # dev server on :5173
vite --host                   # listen on 0.0.0.0 (LAN / containers)
vite --port 3000 --strictPort # fail instead of picking another port
vite --open                   # open the browser on start
vite --force                  # re-run dependency pre-bundling
vite --mode staging           # load .env.staging
vite -c vite.other.config.ts  # use another config file

vite build                    # production build → dist/
vite build --mode staging --sourcemap
vite build --watch            # rebuild on change
vite build --ssr src/entry-server.ts

vite preview --port 8080      # serve dist/
```

CODE EXPLAINED

- → `--host` — expose to other devices; prints a Network URL
- → `--strictPort` — exits if the port is taken
- → `--force` — ignores the `node_modules/.vite` cache
- → `-c` / `--config` — alternate config file path
- → `--ssr` — builds a server entry instead of the client

COMMON FLAGS (ALL COMMANDS)

FLAG	EFFECT
`-c, --config <file>`	Config file to use
`--base <path>`	Public base path
`-m, --mode <mode>`	Mode / env file set
`-l, --logLevel <lvl>`	info | warn | error | silent
`--clearScreen false`	Keep previous terminal output
`-d, --debug [feat]`	Debug logs (e.g. hmr)
`--configLoader <l>`	bundle | runner | native

BUILD FLAGS

FLAG	EFFECT
`--outDir <dir>`	Output folder (dist)
`--sourcemap [type]`	true | inline | hidden
`--minify [m]`	false | oxc | terser
`--emptyOutDir`	Clear outDir outside root
`--manifest`	Write .vite/manifest.json
`-w, --watch`	Rebuild on changes
`--app`	Build all environments

DEV SERVER SHORTCUTS

KEY	ACTION
`h + enter`	Show help
`r + enter`	Restart server
`u + enter`	Show server URLs
`o + enter`	Open in browser
`c + enter`	Clear console
`q + enter`	Quit

⚠ COMMON MISTAKES

- Passing flags to an npm script without `--` (`npm run dev --host`) — npm eats them; use `npm run dev -- --host`
- Using `vite preview` as a production server — it's for local checks only
- Forgetting `--host` inside Docker — the server binds to localhost and is unreachable
- Expecting `--mode production` in dev to minify — mode only changes env and `import.meta.env`

✓ TIPS & BEST PRACTICES

- Bake common flags into scripts: `"dev": "vite --host --open"`
- Use `vite --debug hmr` to see why HMR does a full reload
- `vite build --profile` writes a CPU profile for slow builds
- Add `--strictPort` when other tools expect a fixed port

QUICK SUMMARY

- `vite` — dev server
- `vite build` — production bundle
- `vite preview` — serve the bundle
- `--mode` / `--host` / `--force` — the flags you'll use most

Config File Basics

VITE.CONFIG.TS

Vite looks for `vite.config.{js,mjs,ts,mts,cjs,cts}` in the project root. Export an object wrapped in `defineConfig()` for type hints, or a function that receives `{ command, mode, isSsrBuild, isPreview }` to change settings per command. The config file runs in Node, so it can read files, env vars and other modules.

NOTES

- TS config works with no extra setup
- Config may be `async` (return a Promise)
- `import.meta.env` is **not** available in the config — use `loadEnv`
- Changing the config restarts the dev server automatically

OBJECT AND FUNCTION FORMS

```ts
import { defineConfig, mergeConfig } from 'vite'
import react from '@vitejs/plugin-react'
import base from './vite.base.config'

export default defineConfig(({ command, mode, isSsrBuild }) => {
  const shared = { plugins: [react()] }
  if (command === 'serve') {
    return mergeConfig(base, { ...shared, server: { port: 3000 } })
  }
  return mergeConfig(base, {
    ...shared,
    base: mode === 'staging' ? '/staging/' : '/',
    build: { sourcemap: isSsrBuild ? false : 'hidden' },
  })
})
```

CODE EXPLAINED

- → `defineConfig` — type help only; returns its argument
- → `command` — `'serve'` (dev) or `'build'` (build and preview)
- → `mode` — `'development'`, `'production'` or `--mode` value
- → `isSsrBuild` / `isPreview` — narrow build and preview cases
- → `mergeConfig(a, b)` — deep merge; arrays concatenate

CONFIG FUNCTION ARGUMENT

FIELD	VALUES
`command`	'serve' | 'build'
`mode`	'development' | 'production' | custom
`isSsrBuild`	true during vite build --ssr
`isPreview`	true during vite preview

CONFIG LOADING

OPTION	BEHAVIOR
`--configLoader bundle`	Default; bundle config with Rolldown
`--configLoader runner`	Load via module runner, no temp file
`--configLoader native`	Native Node import (fast, no TS on old Node)
`-c path`	Pick a non-default config file
`loadConfigFromFile()`	Read a config from the JS API

⚠ COMMON MISTAKES

- Reading `import.meta.env.VITE_X` in the config — it's undefined there; call `loadEnv(mode, process.cwd())`
- Spreading two configs with `...` — nested keys like `server` get replaced, not merged; use `mergeConfig`
- Passing a function to `mergeConfig` — it only accepts objects; resolve the function first
- Using `__dirname` in an ESM config without defining it — use `import.meta.dirname` (Node 20.11+)

✓ TIPS & BEST PRACTICES

- Always wrap the export in `defineConfig` for autocomplete
- Branch on `command` rather than `mode` for dev-vs-build differences
- Keep shared settings in a base file and `mergeConfig` per environment
- Use `fileURLToPath(new URL('./src', import.meta.url))` for portable paths

QUICK SUMMARY

- `defineConfig({...})` — typed config object
- `defineConfig(({ command, mode }) => ...)` — conditional
- `mergeConfig(a, b)` — deep merge configs
- `loadEnv()` — env vars inside the config

Shared Options

TOP-LEVEL OPTIONS

Shared options sit at the top level of the config and apply to dev, build and preview alike. The ones you'll touch most are `base` (the public path the app is served from), `define` (compile-time constants) and `publicDir`. Server-, build- and preview-specific settings live under their own keys.

NOTES

- `base` must start and end with `/` (or be `./` for relative)
- `define` values are replaced as raw code, so strings need `JSON.stringify`
- `publicDir: false` turns the public folder off
- `appType`: `'spa'` (default), `'mpa'` or `'custom'`

SHARED OPTIONS IN USE

```
import { defineConfig } from 'vite'

export default defineConfig({
  root: 'src',                          // folder holding index.html
  base: '/my-app/',                     // served from /my-app/
  publicDir: '../static',               // relative to root
  cacheDir: '../node_modules/.vite',
  envDir: '..',                         // where .env files live
  appType: 'spa',                       // HTML fallback for routes
  define: {
    __APP_VERSION__: JSON.stringify('1.4.2'),
    __DEV_TOOLS__: false,
  },
  logLevel: 'info',
  clearScreen: false,
})
```

CODE EXPLAINED

→ `root` — moves where `index.html` and relative paths start

→ `base` — prefix for every asset URL; also `import.meta.env.BASE_URL`

→ `define` — find-and-replace at transform time

→ `appType: 'spa'` — unknown routes return `index.html` in dev

→ `clearScreen: false` — keeps logs from other tools visible

SHARED OPTION REFERENCE

OPTION	DEFAULT	PURPOSE
`root`	process.cwd()	Project root with index.html
`base`	'/'	Public base path; './' for relative
`mode`	per command	Overrides development / production
`define`	{}	Global constant replacements
`publicDir`	'public'	Static files copied as-is
`cacheDir`	'node_modules/.vite'	Pre-bundle cache location
`envDir / envPrefix`	root / 'VITE_'	Env file folder / exposed prefix
`logLevel`	'info'	info · warn · error · silent
`devtools`	false	Enable Vite Devtools (Vite 8)

⚠ COMMON MISTAKES

- `define: { API: 'https://x' }` — inserted unquoted and breaks the code; wrap with `JSON.stringify`
- Setting `base: 'my-app'` without slashes — produces broken asset URLs
- Using `define` for secrets — the value ends up in the shipped JS
- Putting `server`-only options at the top level where they're ignored

✓ TIPS & BEST PRACTICES

- Declare `define` globals in a `.d.ts`: `declare const __APP_VERSION__: string`
- Use `base: './'` for builds opened from arbitrary subfolders
- Prefer `import.meta.env.VITE_*` over `define` for config that varies by environment
- Set `clearScreen: false` when running Vite alongside other watchers

QUICK SUMMARY

- `base` — public path prefix
- `define` — compile-time constants (stringify!)
- `root` / `publicDir` / `envDir` — where things live
- `appType` — spa · mpa · custom

Resolve & Path Aliases

MODULE RESOLUTION

The `resolve` options control how import specifiers become files. **Aliases** map short prefixes like `@/` to real folders. Vite 8 can also read the `paths` from your `tsconfig.json` directly with `resolve.tsconfigPaths: true`, so TypeScript and Vite stay in sync without a plugin.

NOTES

- `tsconfigPaths` is off by default (small perf cost)
- Alias targets should be **absolute** paths
- Vite 8 always respects your `mainFields` order (no format sniffing)
- `customResolver` on aliases is deprecated in Vite 8

ALIASES TWO WAYS

```ts
import { defineConfig } from 'vite'
import { fileURLToPath, URL } from 'node:url'

export default defineConfig({
  resolve: {
    alias: {
      '@': fileURLToPath(new URL('./src', import.meta.url)),
    },
    // or: read "paths" from tsconfig.json (Vite 8)
    // tsconfigPaths: true,
    dedupe: ['react', 'react-dom'],
    conditions: ['custom', 'module', 'browser', 'development|production'],
  },
})

// src/main.ts
import Button from '@/components/Button.vue'
```

CODE EXPLAINED

- → `alias` object — prefix → absolute path
- → `fileURLToPath(new URL(...))` — portable ESM path
- → `tsconfigPaths: true` — no duplicate alias config
- → `dedupe` — force one copy of a package (linked libs)
- → `conditions` — package.json `exports` conditions to match

RESOLVE.* OPTIONS

OPTION	PURPOSE
`alias`	Object or [{ find, replacement }]
`tsconfigPaths`	Use tsconfig paths (Vite 8)
`dedupe`	Single copy of listed deps
`conditions`	Export conditions for client
`mainFields`	package.json fields to try
`extensions`	Extensions tried for bare paths

MATCHING TSCONFIG.JSON

KEY	VALUE
`baseUrl`	"." (optional with paths)
`paths`	{ "@/*": ["./src/*"] }
`moduleResolution`	"bundler"
`allowImportingTsExtensions`	true (optional)

⚠ COMMON MISTAKES

- Defining `@` in `vite.config` but not in `tsconfig.json` — Vite builds while the editor shows errors
- Using a relative alias target like `'./src'` — behaves differently per importer; use absolute paths
- Omitting extensions in imports and relying on `resolve.extensions` for `.vue` — slower and discouraged
- Aliasing a bare package name and breaking that package's own imports

✓ TIPS & BEST PRACTICES

- Prefer `resolve.tsconfigPaths: true` in TS projects so there's one source of truth
- Use array form `{ find: /^~(.*)$/, replacement: '$1' }` for regex aliases
- Add `dedupe: ['react']` when a linked package bundles its own React
- Import with explicit extensions (`./util.ts` style is fine with `allowImportingTsExtensions`)

QUICK SUMMARY

- `resolve.alias` — `@` → `/abs/src`
- `resolve.tsconfigPaths` — reuse tsconfig paths
- `resolve.dedupe` — one copy of a dependency
- `resolve.conditions` — pick package export variants

Env Variables & Modes

HOW ENV WORKS

Vite exposes environment values on `import.meta.env`. Only variables prefixed with **`VITE_`** reach client code, and they're replaced statically at build time, so they end up in the shipped JS. The **mode** (`development` for `vite`, `production` for `vite build`, or whatever `--mode` says) decides which `.env` files load.

NOTES

- Values are always **strings** (`'true'`, not `true`)
- Existing shell env vars beat `.env` files
- `.local` files are git-ignored by convention
- Use `$OTHER_VAR` inside `.env` values to expand (dotenv-expand)

FILES AND CLIENT CODE

```
# .env                  always loaded
# .env.local            always, git-ignored
# .env.[mode]           only in that mode
# .env.[mode].local     only in that mode, git-ignored

# .env.staging
VITE_API_URL=https://staging.api.example.com
VITE_FEATURE_X=true
DB_PASSWORD=secret      # no VITE_ prefix → not exposed

# Type it — src/vite-env.d.ts
interface ImportMetaEnv {
  readonly VITE_API_URL: string
  readonly VITE_FEATURE_X: string
}
```

READING ENV IN CODE AND CONFIG

```
// client code
fetch(`${import.meta.env.VITE_API_URL}/users`)
if (import.meta.env.DEV) console.log('dev')
const on = import.meta.env.VITE_FEATURE_X === 'true'

// vite.config.ts (no import.meta.env here)
import { defineConfig, loadEnv } from 'vite'
export default defineConfig(({ mode }) => {
  const env = loadEnv(mode, process.cwd(), '')
  return {
    define: { __API__: JSON.stringify(env.API) },
    server: { port: Number(env.PORT) || 5173 },
  }
})
```

BUILT-IN VALUES

KEY	VALUE
`import.meta.env.MODE`	Current mode string
`import.meta.env.BASE_URL`	The `base` option
`import.meta.env.PROD`	true in production builds
`import.meta.env.DEV`	true in dev (opposite of PROD)
`import.meta.env.SSR`	true when running on the server

MODES

COMMAND	DEFAULT MODE
`vite`	development
`vite build`	production
`vite build --mode staging`	staging (+ .env.staging)
`vite preview`	production
`NODE_ENV`	Set separately from mode

⚠ COMMON MISTAKES

- Putting secrets in `VITE_*` vars — they're inlined into public JS
- Comparing `import.meta.env.VITE_FLAG === true` — the value is the string `'true'`
- Building dynamic keys like `import.meta.env[name]` — static replacement only works with literal property names
- Assuming `--mode staging` sets `NODE_ENV=staging` — `NODE_ENV` stays `production` for builds

✓ TIPS & BEST PRACTICES

- Commit `.env.example` with every key and dummy values
- Change the prefix with `envPrefix: ['VITE_', 'PUBLIC_']` if needed
- Keep `.env*.local` in `.gitignore`
- Use `loadEnv(mode, root, '')` to read non-prefixed vars in the config

QUICK SUMMARY

- `VITE_` prefix — only these reach the browser
- `.env.[mode].local` — highest-priority file
- `import.meta.env.MODE / DEV / PROD` — built-ins
- `loadEnv()` — env access inside vite.config

TypeScript in Vite

TRANSPILE ONLY

Vite compiles `.ts` and `.tsx` with **Oxc**, which strips types file by file and never type-checks. That keeps dev fast, but you need `tsc` (or `vue-tsc`) in a separate step to catch type errors. Because each file is compiled alone, your `tsconfig` must use settings that work without whole-program information.

NOTES

- Vite 8 replaced the `esbuild` option with `oxc`
- `emitDecoratorMetadata` is supported natively in Vite 8
- `const enum` across files and namespaces merging aren't supported
- Use `import type` for type-only imports

TSCONFIG.JSON (APP)

```jsonc
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "ESNext",
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "verbatimModuleSyntax": true,
    "useDefineForClassFields": true,
    "noEmit": true,
    "jsx": "react-jsx",
    "strict": true,
    "types": ["vite/client"]
  },
  "include": ["src"]
}
```

TYPE CHECKING AND OXC OPTIONS

```ts
// package.json scripts
// "build":     "tsc -b && vite build"
// "check": "tsc --noEmit --watch"

// vite.config.ts
import { defineConfig } from 'vite'
export default defineConfig({
  oxc: {
    jsx: { importSource: 'preact' },
    define: { __TEST__: 'false' },
  },
})

// src/vite-env.d.ts
/// <reference types="vite/client" />
```

REQUIRED / RECOMMENDED TSCONFIG

OPTION	WHY
`isolatedModules`	Flags code that needs whole-program info
`verbatimModuleSyntax`	Forces `import type` for types
`moduleResolution: bundler`	Matches how Vite resolves
`noEmit`	Vite produces the JS, not tsc
`useDefineForClassFields`	Match standard class fields
`types: ["vite/client"]`	Types for env, assets, HMR

WHAT VITE/CLIENT TYPES

IMPORT / API	TYPE
`import.meta.env`	ImportMetaEnv
`import.meta.hot`	ViteHotContext
`import.meta.glob`	Glob import function
`'./x.png'`	string URL
`'./x.svg?raw'`	string
`'./w?worker'`	Worker constructor

⚠ COMMON MISTAKES

- Thinking `vite build` will fail on type errors — it won't; add `tsc -b &&` to the build script
- Exporting a type without `export type` — breaks under isolated compilation
- Leaving out `vite/client` types — `import.meta.env` and `.png` imports show errors
- Keeping old `esbuild: { ... }` options and assuming they're ignored — Vite 8 converts them, but move them to `oxc`

✓ TIPS & BEST PRACTICES

- Run `tsc --noEmit --watch` in a second terminal, or use vite-plugin-checker for overlay errors
- Use `vue-tsc` for Vue and `svelte-check` for Svelte
- Split configs: `tsconfig.app.json` and `tsconfig.node.json` (for vite.config)
- Override a client type by adding your own `declare module '*.svg'` before the reference

QUICK SUMMARY

- `Oxc` — strips types, no checking
- `tsc -b && vite build` — check then build
- `isolatedModules` + `verbatimModuleSyntax` — required style
- `oxc` option — replaces `esbuild` in Vite 8

Dev Server Options

THE SERVER BLOCK

The `server` block configures the development server: where it listens, whether it opens a browser, HTTPS, file-system access and which hostnames it accepts. Defaults are safe for local work: it listens on `localhost:5173`, blocks unknown `Host` headers, and only serves files inside the workspace root.

NOTES

- `server.allowedHosts` guards against DNS rebinding
- `server.fs.strict` blocks files outside the allow list
- `server.forwardConsole` (Vite 8) pipes browser logs to the terminal
- `preview.*` mirrors most of these for `vite preview`

COMMON SERVER SETUP

```ts
import { defineConfig } from 'vite'
import basicSsl from '@vitejs/plugin-basic-ssl'

export default defineConfig({
  plugins: [basicSsl()],              // quick self-signed HTTPS
  server: {
    host: true,                       // 0.0.0.0 — LAN + containers
    port: 3000,
    strictPort: true,
    open: '/dashboard',
    allowedHosts: ['.ngrok.app', 'dev.local'],
    fs: { allow: ['..'] },            // monorepo: serve parent folder
    watch: { usePolling: true },      // Docker / WSL mounts
    forwardConsole: true,             // browser console → terminal
  },
})
```

CODE EXPLAINED

- → `host: true` — listen on every interface
- → `open` — `true` or a path to open
- → `allowedHosts` — leading `.` allows subdomains; `true` allows all
- → `fs.allow` — folders Vite may serve from
- → `watch.usePolling` — needed where file events don't arrive

SERVER.* REFERENCE

OPTION	DEFAULT	PURPOSE
`host`	'localhost'	true / '0.0.0.0' to expose
`port`	5173	Next free port unless strictPort
`strictPort`	false	Exit if port busy
`open`	false	Open browser (true or path)
`https`	undefined	{ key, cert } or basic-ssl plugin
`allowedHosts`	[]	Extra Host headers to accept
`forwardConsole`	auto	Browser logs to terminal

⚠ COMMON MISTAKES

- Fixing "Blocked request. This host is not allowed" with `allowedHosts: true` on a public network — list the exact hosts instead
- Opening a file from a sibling package and getting "outside of Vite serving allow list" — add it to `server.fs.allow`
- Changing files in Docker with no reload — enable `watch.usePolling`
- Expecting `server.port` to apply to `vite preview` — use `preview.port`

✓ TIPS & BEST PRACTICES

- Use `@vitejs/plugin-basic-ssl` or mkcert for HTTPS-only browser APIs
- Set `server.origin` when a backend serves the HTML and Vite serves assets
- Turn on `forwardConsole` when debugging mobile devices from the terminal
- Keep `fs.strict: true`; widen `fs.allow` only as needed

QUICK SUMMARY

- `server.host` / `port` — where it listens
- `server.allowedHosts` — accepted Host headers
- `server.fs.allow` — servable folders
- `server.forwardConsole` — browser logs in terminal

Proxy, CORS & Headers

DEV PROXY

`server.proxy` forwards matching requests from the dev server to another backend, so your front end can call `/api/...` on the same origin and skip CORS entirely during development. Keys are path prefixes, or regexes when they start with ^. For the opposite case (another site calling Vite) use `server.cors`.

NOTES

- The proxy exists only in `vite` and `vite preview`, not in your build
- Production needs the same routing in your real server or CDN
- `preview.proxy` defaults to `server.proxy`
- Vite's default CORS only allows localhost origins

PROXY PATTERNS

```
export default defineConfig({
  server: {
    proxy: {
      '/api': 'http://localhost:8080',                  // shorthand
      '/auth': {
        target: 'https://auth.example.com',
        changeOrigin: true,                             // Host → target
      },
      '/v2': {
        target: 'http://localhost:9000',
        rewrite: (path) => path.replace(/^\/v2/, ''),   // /v2/x → /x
      },
      '/socket.io': { target: 'ws://localhost:5174', ws: true },
    },
    cors: { origin: 'https://app.example.com' },
    headers: { 'Cross-Origin-Opener-Policy': 'same-origin' },
  },
})
```

CODE EXPLAINED

→ String value — target for that prefix, path kept

→ `changeOrigin` — rewrites the `Host` header for vhosts

→ `rewrite` — change the path before forwarding

→ Keys starting with `^` are treated as regexes

→ `ws: true` — proxy WebSocket upgrades too

PROXY OPTIONS

OPTION	PURPOSE
`target`	Backend URL
`changeOrigin`	Send target's host as Host
`rewrite(path)`	Rewrite the forwarded path
`ws`	Proxy WebSockets
`secure`	false = accept bad TLS certs
`configure(proxy, opts)`	Hook into proxy events
`bypass(req, res)`	Return a path to skip proxy

CORS & HEADERS

OPTION	EFFECT
`cors: true`	Allow any origin (avoid)
`cors: { origin }`	Allow specific origins
`cors: false`	Disable CORS handling
`headers`	Add headers to every response
`preview.headers`	Same, for vite preview

⚠ COMMON MISTAKES

- Forgetting `changeOrigin: true` for name-based hosts — the backend sees `localhost` and returns 404
- Writing the `rewrite` regex without the `^` anchor and stripping `/api` from the middle of paths
- Shipping code that relies on the proxy — production requests hit your static host and 404
- Setting `cors: true` while the dev server is exposed with `--host` — any site can read your dev files

✓ TIPS & BEST PRACTICES

- Keep API calls relative (`/api/users`) so the same code works behind a production reverse proxy
- Log proxy traffic with `configure: (proxy) => proxy.on('proxyReq', ...)`
- Set COOP/COEP headers in `server.headers` to test `SharedArrayBuffer` locally
- Use a regex key to proxy everything except Vite's own paths

QUICK SUMMARY

- `server.proxy['/api']` — forward to a backend
- `changeOrigin` / `rewrite` / `ws` — key options
- `server.cors` — who may call the dev server
- `server.headers` — extra response headers

Hot Module Replacement

HOW HMR WORKS

HMR swaps changed modules into the running page without a reload, keeping state. Framework plugins (React Fast Refresh, Vue, Svelte) handle it for you; `import.meta.hot` is the low-level API for libraries and plain JS. When a changed module has no accepting boundary, Vite falls back to a full page reload.

NOTES

- `import.meta.hot` is `undefined` in production — always guard it
- The `if (import.meta.hot)` block is removed from builds
- CSS files hot-update automatically
- Vite 8 removed passing URLs to `hot.accept`; pass ids

SELF-ACCEPTING MODULE

```
// counter.js
export let count = 0
const timer = setInterval(() => count++, 1000)

if (import.meta.hot) {
  import.meta.hot.data.count ??= 0              // persists across updates
  import.meta.hot.accept((newModule) => {       // this file is a boundary
    if (newModule) console.log('updated', newModule.count)
  })
  import.meta.hot.dispose((data) => {           // clean up old version
    clearInterval(timer)
    data.count = count
  })
}
```

CODE EXPLAINED

→ `accept(cb)` — module handles its own updates

→ `dispose(cb)` — undo side effects before replacement

→ `data` — object kept between versions of the module

→ Block is stripped from production builds

IMPORT.META.HOT API

METHOD	PURPOSE
`accept()`	Self-accept, no callback
`accept(deps, cb)`	Accept one or more deps
`dispose(cb)`	Cleanup before swap
`invalidate(msg)`	Give up; propagate to importers
`on / off(event, cb)`	Subscribe to events

BUILT-IN EVENTS

EVENT	WHEN
`vite:beforeUpdate`	Update about to apply
`vite:afterUpdate`	Update applied
`vite:beforeFullReload`	Page about to reload
`vite:error`	Transform error (overlay)

SERVER.HMR

OPTION	USE
`overlay: false`	Hide the error overlay
`port / clientPort`	WS port behind proxies
`host / protocol`	WS host, ws or wss
`hmr: false`	Disable HMR entirely

⚠ COMMON MISTAKES

- Calling `import.meta.hot.accept()` without a guard — crashes in production where `hot` is undefined
- Using `const hot = import.meta.hot` and calling `hot.accept` — Vite needs the literal `import.meta.hot.accept(...)` to detect boundaries
- Exporting non-components from a React component file — Fast Refresh falls back to a full reload

✓ TIPS & BEST PRACTICES

- Run `vite --debug hmr` to see which module triggered a full reload
- Use `invalidate()` when an update can't be applied safely
- Behind a reverse proxy set `server.hmr.clientPort` to the public port
- Keep React component files exporting components only

QUICK SUMMARY

- `import.meta.hot.accept()` — mark an HMR boundary
- `dispose()` + `data` — clean up and carry state
- `vite:*` events — hook into the update cycle
- `server.hmr` — WebSocket and overlay settings

Dependency Pre-Bundling

WHY PRE-BUNDLE

On first start, Vite scans your source for bare imports and **pre-bundles** those dependencies with Rolldown into `node_modules/.vite/deps`. This converts CommonJS packages to ESM and collapses packages made of hundreds of files (like `lodash-es`) into one request each. Results are cached and reused until your lockfile, config or patches change.

NOTES

- Dev-only; the build bundles deps directly
- Vite 8 renamed `optimizeDeps.esbuildOptions` → `rolldownOptions`
- Deps found late trigger a re-bundle and page reload
- Browser caches deps with `max-age=31536000,immutable`

OPTIMIZEDEPS IN PRACTICE

```
export default defineConfig({
  optimizeDeps: {
    include: [
      'lodash-es',                    // many small files
      'my-cjs-lib',                   // CJS → ESM
      'linked-pkg > nested-cjs-dep',  // dep of a linked package
    ],
    exclude: ['@my-org/ui-kit'],      // linked ESM source; let HMR work
    entries: ['src/main.ts', 'src/admin/*.html'],
    holdUntilCrawlEnd: true,          // wait for full crawl on cold start
    rolldownOptions: {                // was esbuildOptions
      resolve: { mainFields: ['module', 'main'] },
    },
  },
})
```

CODE EXPLAINED

- → `include` — force a dep into pre-bundling
- → `a > b` — target a nested dependency
- → `exclude` — serve a package as raw source
- → `entries` — where the scanner starts
- → `rolldownOptions` — Rolldown settings for the optimizer

WHEN TO USE WHICH

SITUATION	FIX
Reload after first visit to a route	Add the late dep to include
CJS dep inside a linked package	include: ['linked > dep']
Linked package edits ignored	exclude it (if it's ESM)
"504 Outdated Optimize Dep"	Reload; or vite --force
Stale dep after patch-package	vite --force
Deps behind dynamic import()	include them explicitly

OPTIMIZEDEPS.*

OPTION	PURPOSE
include / exclude	Force in / keep out
entries	Scanner entry globs
force	Ignore the cache
noDiscovery	Only bundle include list
holdUntilCrawlEnd	Fewer reloads on cold start
rolldownOptions	Optimizer bundler options

⚠ COMMON MISTAKES

- Excluding a CommonJS package — the browser can't load raw CJS and you get "does not provide an export named …"
- Deleting `node_modules` to clear the cache — `vite --force` or removing `node_modules/.vite` is enough
- Ignoring repeated "new dependencies optimized" logs — each one costs a reload; add them to `include`
- Copying old `esbuildOptions.plugins` — Rolldown support for those is partial; port them to Rolldown plugins

✓ TIPS & BEST PRACTICES

- Watch the terminal on first load and add any late-discovered deps to `include`
- In monorepos, `exclude` your linked ESM packages so edits hot-reload
- Use `DEBUG=vite:deps vite` to see what the optimizer is doing
- Commit the `include` list so every developer gets the same cold start

QUICK SUMMARY

- `optimizeDeps.include` — pre-bundle these
- `optimizeDeps.exclude` — serve as source
- `vite --force` — rebuild the dep cache
- `node_modules/.vite/deps` — where results live

Static Assets

IMPORTING ASSETS

Importing an image, font or other file gives you its **final URL**: in dev it points at the source, in the build it's a hashed file in `dist/assets/`. Small files are inlined as base64 data URLs. Query suffixes change what the import returns, and `public/` holds files you reference by a fixed absolute path instead.

NOTES

- `assetsInlineLimit` default is `4096` bytes
- SVGs under the limit inline as URL-encoded text
- `new URL('./x.png', import.meta.url)` works without an import
- Add extra types with `assetsInclude: ['**/*.gltf']`

ASSET IMPORTS

```
import logoUrl from './logo.png'             // '/assets/logo-B2xF4k.png'
import iconUrl from './icon.svg?url'         // force URL, never inline
import shader from './glow.glsl?raw'         // file contents as string
import tinyPng from './dot.png?inline'       // always a data: URL
import bigSvg from './map.svg?no-inline'     // never inline

img.src = logoUrl

// Dynamic path without an import
const flag = new URL(`./flags/${code}.svg`, import.meta.url).href

// Public folder: absolute path, no import, no hash
favicon.href = '/favicon.ico'

// CSS url() is rewritten too:  background: url('./bg.jpg')
```

CODE EXPLAINED

- → Default import — final public URL as a string
- → `?url` — URL even for files that would be inlined
- → `?raw` — contents as a string (shaders, SVG markup)
- → `?inline` / `?no-inline` — force inlining on or off
- → `new URL(..., import.meta.url)` — static template literal only

IMPORT SUFFIXES

SUFFIX	RETURNS
(none)	URL (or data URL if small)
?url	URL string
?raw	File contents string
?inline	data: URL
?no-inline	URL, never inlined
?worker	Worker constructor (sheet 16)
?init	Wasm init function (sheet 16)

PUBLIC/ VS SRC/ASSETS/

	PUBLIC/	SRC/ASSETS/
Reference	/file.ext	import x from './file'
Hashed	✖	✔
Inlined if small	✖	✔
Missing file error	✖ (404)	✔ (build fails)
Good for	robots.txt, favicon	Everything else

⚠ COMMON MISTAKES

- Hard-coding `'/src/assets/logo.png'` strings — they work in dev but break after the build
- Importing from `public/` — reference public files by `/name` only; importing them warns and copies twice
- Using fully dynamic `new URL(variable, import.meta.url)` — Vite can only see template literals with a static prefix
- Expecting `?raw` content to be transformed — it's the untouched file text

✓ TIPS & BEST PRACTICES

- Set `build.assetsInlineLimit: 0` to never inline (e.g. strict CSP)
- Use `?url` for SVGs you set as `<img src>` to avoid huge inlined strings
- Prefix public paths with `import.meta.env.BASE_URL` when `base` isn't `/`
- Declare extra suffix types in `vite-env.d.ts` if you add custom ones

QUICK SUMMARY

- `import url from './x.png'` — hashed URL
- `?raw` / `?url` / `?inline` — change the result
- `public/` — fixed paths, no processing
- `assetsInlineLimit` — inline threshold (4 KiB)

Glob Imports

IMPORT.META.GLOB

`import.meta.glob()` imports many files at once using a glob pattern. Vite expands it at compile time into an object whose keys are file paths. By default each value is a **lazy loader** (a function returning a dynamic `import()`), which creates a separate chunk per file. With `eager: true` the files are imported up front instead.

NOTES

- Arguments must be **string literals**, not variables
- Patterns are relative (`./`), absolute (`/`) or aliased
- Prefix with `!` to exclude files
- Vite 8.1 adds `caseSensitive: false`

LAZY, EAGER AND NAMED

```
// Lazy: { './pages/About.tsx': () => import('./pages/About.tsx'), ... }
const pages = import.meta.glob('./pages/*.tsx')
const mod = await pages['./pages/About.tsx']()

// Eager: modules imported immediately
const locales = import.meta.glob('./locales/*.json', { eager: true })

// Only the default export, eager
const icons = import.meta.glob('./icons/*.svg', {
  eager: true, query: '?raw', import: 'default',
})                                // { './icons/x.svg': '<svg…>' }

// Several patterns, with exclusion and a typed result
const tests = import.meta.glob<{ run: () => void }>(
  ['./specs/**/*.ts', '!**/*.skip.ts'],
)
```

CODE EXPLAINED

- → Default — values are `() => Promise<Module>`
- → `eager: true` — values are the modules themselves
- → `import: 'default'` — pick one export (tree-shakes the rest)
- → `query: '?raw'` — apply an import suffix to every match
- → Generic `<T>` — types the module values

OPTIONS

OPTION	EFFECT
`eager`	Import immediately, no chunks
`import`	Named export to take ('default', 'setup')
`query`	Suffix like '?raw' or '?url'
`base`	Base folder for result keys
`exhaustive`	Also search node_modules / dotfiles
`caseSensitive`	false = ignore case (Vite 8.1)

PATTERN EXAMPLES

PATTERN	MATCHES
`./dir/*.ts`	One level
`./dir/**/*.ts`	Any depth
`./dir/*.{png,jpg}`	Several extensions
`!**/*.test.ts`	Exclude matches
`/src/routes/*.vue`	From project root
`@/plugins/*.ts`	Through an alias

⚠ COMMON MISTAKES

- Building the pattern from a variable — glob arguments must be literal strings or arrays of literals
- Using `eager: true` for large route sets — everything lands in the main bundle; keep routes lazy
- Expecting keys to be file names — keys are the relative paths you wrote, such as `./pages/Home.tsx`
- Matching the file that calls the glob — it can import itself; add a `!` exclusion

✓ TIPS & BEST PRACTICES

- Build a route table from `import.meta.glob('./pages/**/*.tsx')` for file-based routing
- Use `import: 'default', eager: true` for small registries like icons or translations
- Use `base` to strip a common folder from the keys
- For one dynamic file, prefer `import(` ./locales/${lang}.json `)` (sheet 17)

QUICK SUMMARY

- `import.meta.glob('./x/*.ts')` — lazy loaders
- `{ eager: true }` — modules up front
- `{ import: 'default', query: '?raw' }` — shape results
- `'!pattern'` — exclude files

JSON, WebAssembly & Web Workers

SPECIAL IMPORTS

Vite handles three special file types out of the box. **JSON** imports become ES modules, and named imports let unused fields be tree-shaken. **WebAssembly** loads with `?init`, and Vite 8.1 adds direct ESM imports of `.wasm` files. **Web Workers** are bundled as their own entry whether you use the `?worker` suffix or the standard `new Worker(new URL(...))` pattern.

NOTES

- `json.stringify` emits `JSON.parse("…")` for large files
- `.wasm?init` works in SSR since Vite 8
- Workers default to `worker.format: 'iife'`
- Use `{ type: 'module' }` for ES module workers

ALL THREE PATTERNS

```
// JSON — default or named (named = tree-shakeable)
import pkg from './package.json'
import { version } from './package.json'

// WebAssembly with ?init
import init from './math.wasm?init'
const instance = await init({ env: { log: console.log } })

// Web Worker — standard syntax (recommended)
const worker = new Worker(new URL('./heavy.ts', import.meta.url),
  { type: 'module' })
worker.postMessage({ n: 40 })

// Web Worker — Vite suffix
import HeavyWorker from './heavy.ts?worker'
import InlineWorker from './heavy.ts?worker&inline'   // base64 blob
```

CODE EXPLAINED

→ Named JSON import — only that field ships

→ `?init` — returns `init(imports?) → Promise<Instance>`

→ `new URL('./w.ts', import.meta.url)` — must be inline in `new Worker()`

→ `?worker` — default export is a Worker constructor

→ `?worker&inline` — worker embedded as base64

WORKER SUFFIXES

IMPORT	RESULT
`?worker`	Worker constructor
`?sharedworker`	SharedWorker constructor
`?worker&inline`	Inlined worker
`?worker&url`	URL of the bundled worker

WORKER.* CONFIG

OPTION	PURPOSE
`format`	'iife' (default) or 'es'
`plugins`	() => plugins for workers
`rolldownOptions`	Bundler options (was rollupOptions)

WASM & JSON OPTIONS

OPTION	PURPOSE
`?init`	Instantiate Wasm
`?url`	URL of the .wasm file
`'./x.wasm'`	Direct ESM import (8.1)
`json.namedExports`	Allow named imports
`json.stringify`	true | false | 'auto'

⚠ COMMON MISTAKES

- Storing the URL in a variable first — `new Worker(url)` isn't detected; write `new Worker(new URL(...))` inline
- Using `import` inside an IIFE worker — set `worker.format: 'es'` and `{ type: 'module' }`
- Importing a `.wasm` file with no suffix before Vite 8.1 — it fails; add `?init` or upgrade for ESM integration

✓ TIPS & BEST PRACTICES

- Prefer the `new URL` worker syntax — it's standard and works in other tools too
- Use `?worker&inline` only for small workers; it inflates the main bundle
- Use Comlink to call worker functions like async methods
- Name JSON imports to keep big config files out of the bundle

QUICK SUMMARY

- `import { version } from './package.json'` — JSON
- `import init from './x.wasm?init'` — Wasm
- `new Worker(new URL('./w.ts', import.meta.url))` — worker
- `worker.format: 'es'` — module workers

Dynamic Imports & Preloading

CODE SPLITTING BY IMPORT

A dynamic `import()` splits code into a separate chunk that loads on demand. Vite supports template-literal paths with variables, turning them into a glob of possible files, and automatically **preloads** a chunk's dependencies in parallel so lazy routes don't waterfall. If a chunk fails to load (often after a new deploy), Vite fires a `vite:preloadError` event you can handle.

NOTES

- Variable paths must start with `./` or `../`
- They must include the file extension
- Variables can only fill a single folder level
- `/* @vite-ignore */` silences warnings for truly runtime paths

DYNAMIC IMPORT PATTERNS

```
// Lazy route / component → own chunk
const Settings = () => import('./pages/Settings.tsx')

// Variable path → Vite globs ./locales/*.json
const messages = await import(`./locales/${lang}.json`)

// Runtime URL Vite shouldn't analyze
const plugin = await import(/* @vite-ignore */ remoteUrl)

// Reload once when chunks 404 after a deploy
window.addEventListener('vite:preloadError', (event) => {
  event.preventDefault()          // stop the error from throwing
  window.location.reload()
})

// vite.config.ts
// build: { modulePreload: { polyfill: false } }
```

CODE EXPLAINED

- → Static string — one chunk per imported file
- → Template literal — all matching files become chunks
- → `@vite-ignore` — leave the import untouched
- → `vite:preloadError` — handle missing chunks
- → `modulePreload.polyfill` — off if you only target modern browsers

VARIABLE IMPORT RULES

PATH	ALLOWED?
`` `./locales/${l}.json` ``	✔
`` `./pages/${a}/index.tsx` ``	✔ one level
`` `${dir}/file.js` ``	✖ must start ./
`` `./locales/${l}` ``	✖ needs extension
`` `./${a}/${b}.js` ``	✖ nested variables
`import(name)`	✖ not analyzable

BUILD.MODULEPRELOAD

OPTION	EFFECT
`true (default)`	Preload deps + polyfill
`{ polyfill: false }`	No modulepreload polyfill
`resolveDependencies`	Filter or reorder preloads
`false`	No automatic preloading

⚠ COMMON MISTAKES

- Using a bare variable `import(path)` — Vite can't know the files, so nothing gets bundled
- Lazy-loading tiny modules — each chunk is an extra request; split at routes and heavy libraries
- Ignoring stale-chunk errors after deploys — users with an old tab get blank screens; handle `vite:preloadError`
- Assuming `@vite-ignore` makes a local file work — it won't be in `dist/`

✓ TIPS & BEST PRACTICES

- Split at route boundaries with `React.lazy` / `defineAsyncComponent`
- Keep old assets on the server for a while after deploys so open tabs still load
- Use `import.meta.glob` when you need the list of files, not just one
- Check chunk sizes with a bundle analyzer after adding lazy imports

QUICK SUMMARY

- `import('./x')` — new chunk on demand
- `` `import(./dir/${v}.ext)` `` — globbed variable import
- `vite:preloadError` — recover from missing chunks
- `build.modulePreload` — preload settings

CSS & CSS Modules

CSS IN VITE

Importing a `.css` file injects it into the page in dev (with HMR) and extracts it into a hashed `.css` file in the build. Files named `*.module.css` are **CSS Modules**: class names get scoped, and the import returns an object mapping your names to the generated ones. `@import` statements are inlined and relative `url()`s are rewritten.

NOTES

- PostCSS runs automatically if a config exists
- `?inline` returns the CSS string without injecting it
- Works for `.module.scss`, `.module.less` too
- Vite 8 build minifies CSS with Lightning CSS

IMPORTING CSS

```
import './global.css'               // global
import styles from './Btn.module.css' // scoped
import { primary } from './Btn.module.css'
import printCss from './print.css?inline'
import themeUrl from './theme.css?url'

btn.className = styles.primary  // 'primary_x7d2a'
btn.className = styles['is-active']

const style = document.createElement('style')
style.textContent = printCss
```

MODULE FILE + CONFIG

```
/* Button.module.css */
.primary { color: white; background: var(--brand); }
.is-active { outline: 2px solid; }
.large { composes: primary; font-size: 1.25rem; }

/* vite.config.ts
css: {
  modules: {
    localsConvention: 'camelCaseOnly', // styles.isActive
    generateScopedName: '[name]__[local]__[hash:base64:5]',
  },
  devSourcemap: true,
} */
```

CSS IMPORT FORMS

IMPORT	RESULT
`import './a.css'`	Injected; side effect only
`import s from './a.module.css'`	Class name map
`import { x } from './a.module.css'`	Named class
`'./a.css?inline'`	CSS string
`'./a.css?url'`	Built file URL

CSS.MODULES OPTIONS

OPTION	PURPOSE
`localsConvention`	camelCase · camelCaseOnly · dashes
`generateScopedName`	Pattern or function
`scopeBehaviour`	'local' (default) or 'global'
`globalModulePaths`	Files treated as global
`hashPrefix`	Salt for hashes

⚠ COMMON MISTAKES

- Naming a file `Button.css` and expecting scoped classes — only `*.module.css` files are modules
- Using `styles.is-active` — dashes need bracket access or `localsConvention`
- Importing CSS with `?inline` and wondering why nothing styles the page — `?inline` never injects
- Relying on CSS import order across lazy chunks — chunk CSS loads when the chunk does

✓ TIPS & BEST PRACTICES

- Use `:global(.name)` inside a module for one unscoped selector
- Enable `css.devSourcemap` to map styles back to source files in DevTools
- Use `composes:` to share styles between module classes
- Use `?inline` for Shadow DOM styles in web components

QUICK SUMMARY

- `import './x.css'` — global, injected
- `*.module.css` — scoped class map
- `?inline` / `?url` — string or URL
- `css.modules` — naming and scoping options

Preprocessors, PostCSS & Lightning CSS

BEYOND PLAIN CSS

Vite supports Sass, Less and Stylus once you install the preprocessor package; there are no Vite plugins to add. **PostCSS** runs automatically when a `postcss.config.*` file exists. Alternatively set `css.transformer: 'lightningcss'` to use **Lightning CSS** for nesting, prefixing and syntax lowering instead of PostCSS (Vite 8.1 closed most gaps; it may become the default).

NOTES

- Install `sass-embedded` (fastest) or `sass`
- Lightning CSS is a regular dependency since Vite 8
- PostCSS plugins don't run when the transformer is Lightning CSS
- `css.lightningcss` takes `targets`, `drafts` and similar options

CONFIG FOR EACH APPROACH

```
// npm i -D sass-embedded   (or less / stylus)
export default defineConfig({
  css: {
    preprocessorOptions: {
      scss: {
        additionalData: `@use "@/styles/vars" as *;`,  // every file
      },
      less: { math: 'always', javascriptEnabled: true },
    },
    preprocessorMaxWorkers: true,     // parallel Sass/Less
    devSourcemap: true,

    // Option B: Lightning CSS instead of PostCSS
    // transformer: 'lightningcss',
    // lightningcss: { drafts: { customMedia: true } },
  },
})
// postcss.config.js → export default { plugins: [autoprefixer()] }
```

CODE EXPLAINED

- → `additionalData` — prepended to every `.scss` file
- → Keys are language names: `scss`, `sass`, `less`, `styl`
- → `preprocessorMaxWorkers` — run compilers in workers
- → `transformer: 'lightningcss'` — replaces PostCSS
- → `postcss.config.js` — auto-detected, no Vite setting

PREPROCESSOR SETUP

LANGUAGE	INSTALL	FILES
Sass (SCSS)	sass-embedded or sass	.scss / .sass
Less	less	.less
Stylus	stylus	.styl / .stylus
PostCSS	postcss + plugins	postcss.config.js
Lightning CSS	built in (Vite 8)	css.transformer

CSS.* OPTIONS

OPTION	PURPOSE
preprocessorOptions	Per-language settings
postcss	Inline config or path
transformer	'postcss' | 'lightningcss'
lightningcss	Lightning CSS options
devSourcemap	CSS sourcemaps in dev

⚠ COMMON MISTAKES

- Putting real CSS rules in `additionalData` — they're duplicated into every file; share variables and mixins only
- Using `@import` in Sass — it's deprecated; switch to `@use` / `@forward`
- Switching to `transformer: 'lightningcss'` and expecting Tailwind's PostCSS plugin to run — use `@tailwindcss/vite` instead
- Installing `node-sass` — unsupported; use `sass-embedded`

✓ TIPS & BEST PRACTICES

- Prefer native CSS nesting and custom properties; you may not need a preprocessor
- Keep `browserslist` in package.json so Autoprefixer and Lightning CSS target the same browsers
- Try `transformer: 'lightningcss'` for faster CSS processing in big apps
- Alias style folders (`@/styles`) so `@use` paths stay short

QUICK SUMMARY

- `npm i -D sass-embedded` — Sass support
- `css.preprocessorOptions.scss` — shared vars
- `postcss.config.js` — auto-loaded
- `css.transformer: 'lightningcss'` — PostCSS alternative

Tailwind & CSS Build Options

TAILWIND + CSS OUTPUT

Tailwind CSS v4 ships a first-party Vite plugin, `@tailwindcss/vite`, that's faster than the PostCSS route and needs no `tailwind.config.js` for most projects. The other half of this sheet covers how Vite outputs CSS in production: per-chunk CSS files, minification (Lightning CSS by default in Vite 8) and the browser target used to lower modern syntax.

NOTES

- Tailwind v4 configures with `@theme` in CSS
- Content sources are detected automatically
- `build.cssCodeSplit: false` → one CSS file
- `build.cssTarget` defaults to `build.target`

TAILWIND V4 SETUP

```
// npm i -D tailwindcss @tailwindcss/vite
// vite.config.ts
import { defineConfig } from 'vite'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [tailwindcss()],
})

// src/main.ts
import './app.css'
```

APP.CSS + BUILD OPTIONS

```
@import "tailwindcss";
@theme {
  --color-brand: oklch(0.62 0.19 260);
  --font-display: "Inter", sans-serif;
}
@source "../node_modules/@my/ui";

/* vite.config.ts → build: {
     cssCodeSplit: true,       one CSS per chunk
     cssMinify: 'lightningcss',
     cssTarget: 'chrome111',
   } */
```

CSS BUILD OPTIONS

OPTION	DEFAULT	EFFECT
`build.cssCodeSplit`	true	CSS split per async chunk
`build.cssMinify`	'lightningcss'	Or 'esbuild' (needs esbuild), false
`build.cssTarget`	build.target	Browsers for CSS lowering
`css.transformer`	'postcss'	Pipeline for dev and build
`build.assetsInlineLimit`	4096	Inline small url() assets

TAILWIND V4 DIRECTIVES

DIRECTIVE	USE
`@import "tailwindcss"`	Load Tailwind
`@theme { --x }`	Design tokens → utilities
`@source "path"`	Extra content folders
`@utility name {}`	Custom utility
`@variant dark`	Apply a variant in CSS

⚠ COMMON MISTAKES

- Following Tailwind v3 guides (`tailwind.config.js` , `@tailwind base`) for a v4 install
- Using both `@tailwindcss/vite` and `@tailwindcss/postcss` — pick one
- Setting `cssTarget` lower than needed — forces extra prefixes and fallbacks
- Turning off `cssCodeSplit` in a large app — every page downloads all CSS

✓ TIPS & BEST PRACTICES

- Add `@source` for class names inside `node_modules` component libraries
- Use `@theme` variables in plain CSS too: `color: var(--color-brand)`
- Set `cssCodeSplit: false` for libraries that ship one stylesheet
- Use the Tailwind IntelliSense editor extension for v4 autocomplete

QUICK SUMMARY

- `@tailwindcss/vite` — Tailwind v4 plugin
- `@import "tailwindcss"` + `@theme` — CSS-first config
- `build.cssCodeSplit` — per-chunk CSS
- `build.cssMinify` — Lightning CSS by default

Build Basics

WHAT A BUILD PRODUCES

`vite build` bundles `index.html` and everything it imports into `dist/`: hashed JS and CSS chunks, processed assets, and copies of `public/`. The defaults target browsers in the **Baseline Widely Available** set. Most tuning happens under the `build` key; bundler-level control lives in `build.rolldownOptions` (sheet 22).

NOTES

- Mode is `production`, so `import.meta.env.PROD` is `true`
- `dist/` is emptied first when it's inside `root`
- File names include a content hash for long-term caching
- Vite 8 target: Chrome/Edge 111, Firefox 114, Safari 16.4

BUILD CONFIG

```ts
export default defineConfig({
  build: {
    outDir: 'dist',
    assetsDir: 'assets',            // inside outDir
    emptyOutDir: true,              // needed if outDir is outside root
    sourcemap: 'hidden',            // maps without //# comment
    manifest: true,                 // dist/.vite/manifest.json
    target: 'baseline-widely-available',
    copyPublicDir: true,
    reportCompressedSize: false,    // faster on big projects
    chunkSizeWarningLimit: 800,     // kB
  },
})
// dist/
//   index.html
//   assets/index-BvX1kq3d.js   assets/index-Dk2m.css   favicon.svg
```

CODE EXPLAINED

- → `outDir` / `assetsDir` — where files land
- → `sourcemap` — `true`, `'inline'` or `'hidden'`
- → `manifest` — map of source entries → built files (backends)
- → `target` — syntax level for output JS
- → `reportCompressedSize` — skip gzip size calc for speed

BUILD.* QUICK REFERENCE

OPTION	DEFAULT	PURPOSE
`outDir`	'dist'	Output folder
`assetsDir`	'assets'	Subfolder for assets
`sourcemap`	false	true · 'inline' · 'hidden'
`manifest / ssrManifest`	false	Write manifest files
`target`	baseline	JS syntax target
`minify`	'oxc'	false · 'terser' (sheet 23)
`write`	true	false = keep in memory
`watch`	null	{} = rebuild on change

SOURCEMAP CHOICES

VALUE	USE WHEN
`false`	Default; smallest output
`true`	Separate .map + comment
`'hidden'`	Upload maps to Sentry, hide from users
`'inline'`	Debug builds only (huge files)

⚠ COMMON MISTAKES

- Serving `dist/` from a subpath without setting `base` — every asset 404s
- Building into a folder outside the project with no `emptyOutDir: true` — stale files pile up
- Shipping `sourcemap: true` when you don't want source visible — use `'hidden'`
- Treating the chunk-size warning as an error — it's advice; split or raise the limit

✓ TIPS & BEST PRACTICES

- Run `vite build && vite preview` before every deploy
- Use `build.manifest` when a server-side framework renders the HTML
- Set `reportCompressedSize: false` in CI to save time
- Pin a custom `target` only when you must support older browsers

QUICK SUMMARY

- `vite build` — bundle into `dist/`
- `build.outDir` / `assetsDir` — output layout
- `build.sourcemap: 'hidden'` — private maps
- `build.manifest` — entry → file mapping

Rolldown Options & Chunking

BUNDLER CONTROL

`build.rolldownOptions` passes settings straight to Rolldown, the bundler. Use it for **multiple HTML entries**, custom output file names, externals and chunk grouping. Vite 8 renamed it from `build.rollupOptions` (the old name still works with a deprecation warning). Rolldown's **`codeSplitting.groups`** replaces `manualChunks` for vendor splitting.

NOTES

- Object-form `manualChunks` is removed in Vite 8
- Function-form `manualChunks` is deprecated
- Group `test` regexes should use `[\\/]` for Windows paths
- Output formats `system` and `amd` are no longer supported

MULTI-PAGE + VENDOR CHUNKS

```ts
import { resolve } from 'node:path'
const r = (p: string) => resolve(import.meta.dirname, p)

export default defineConfig({
  build: {
    rolldownOptions: {
      input: { main: r('index.html'), admin: r('admin/index.html') },
      external: ['some-cdn-lib'],
      output: {
        entryFileNames: 'js/[name]-[hash].js',
        chunkFileNames: 'js/chunks/[name]-[hash].js',
        assetFileNames: 'static/[name]-[hash][extname]',
        codeSplitting: {
          groups: [
            { name: 'react', test: /node_modules[\\/]react/, priority: 2 },
            { name: 'vendor', test: /node_modules/, priority: 1 },
          ],
        },
      },
    },
  },
})
```

CODE EXPLAINED

- → `input` object — one entry per HTML page
- → `external` — leave imports unbundled
- → `*FileNames` — placeholders `[name]` `[hash]` `[ext]`
- → `codeSplitting.groups` — named chunk groups
- → `priority` — higher groups claim modules first

CODESPLITTING GROUP FIELDS

FIELD	PURPOSE
`name`	Chunk name, or (id) => name
`test`	RegExp / string / (id) => bool
`priority`	Higher matches first (default 0)
`minSize / maxSize`	Byte limits; split above max
`minShareCount`	Entries that must share a module

VITE 7 → VITE 8 NAMES

BEFORE	NOW
`build.rollupOptions`	build.rolldownOptions
`worker.rollupOptions`	worker.rolldownOptions
`output.manualChunks {}`	Removed → codeSplitting
`output.manualChunks fn`	Deprecated
`watch.chokidar`	watch.watcher

⚠ COMMON MISTAKES

- Copying a Vite 7 `manualChunks: { vendor: [...] }` object — it no longer works; rewrite as `codeSplitting.groups`
- Adding extra HTML pages without listing them in `input` — only `index.html` builds
- Using `/node_modules\/react/` — misses Windows paths; use `[\\/]`
- Putting every dependency in one vendor chunk — one change busts the whole cache

✓ TIPS & BEST PRACTICES

- Split rarely changing, heavy libs (React, charting) into their own groups
- Use `import.meta.dirname` instead of `__dirname` in ESM configs
- Mark CDN-loaded libraries `external` and add them via `<script>` or an import map
- Measure with a bundle analyzer before and after changing chunking

QUICK SUMMARY

- `build.rolldownOptions` — Rolldown settings (was rollupOptions)
- `input: { a, b }` — multi-page apps
- `output.codeSplitting.groups` — vendor chunks
- `external` — don't bundle these imports

Minification & Bundle Size

SHRINKING THE OUTPUT

Vite 8 minifies JavaScript with the **Oxc minifier** and CSS with **Lightning CSS**. You can still choose Terser, or esbuild if you install it, but both are slower. Beyond minification, bundle size comes down to tree-shaking (ESM imports, side-effect-free packages), splitting heavy code into lazy chunks, and checking what actually ships.

NOTES

- `build.minify: 'esbuild'` is deprecated and needs `esbuild` installed
- Property mangling isn't supported by Oxc
- Tree-shaking needs ESM; CJS deps shake poorly
- Vite 8.1 adds experimental `build.chunkImportMap`

MINIFY AND ANALYZE

```ts
import { visualizer } from 'rollup-plugin-visualizer'

export default defineConfig({
  plugins: [visualizer({ filename: 'stats.html', gzipSize: true })],
  build: {
    minify: 'oxc',                        // default; or 'terser' / false
    cssMinify: 'lightningcss',            // default
    rolldownOptions: {
      output: {
        minify: {
          compress: { dropConsole: true, dropDebugger: true },
        },
      },
      treeshake: { moduleSideEffects: 'no-external' },
    },
    chunkSizeWarningLimit: 600,
  },
})
```

CODE EXPLAINED

- → `minify: 'oxc'` — fast default JS minifier
- → `output.minify.compress.dropConsole` — strip `console.*` (was `esbuild.drop`)
- → `dropDebugger` — remove `debugger` statements
- → `treeshake` — Rolldown tree-shaking options
- → `visualizer` — treemap of what's in each chunk

MINIFIER OPTIONS

VALUE	NOTES
`'oxc'`	Default JS minifier (Vite 8)
`'terser'`	npm i -D terser; slowest, most options
`'esbuild'`	Deprecated; npm i -D esbuild
`false`	No minification (debugging)
`cssMinify: 'lightningcss'`	Default CSS minifier

SIZE CHECKLIST

CHECK	FIX
`Large lib in main chunk`	Lazy import() it
`Whole lodash bundled`	Use lodash-es + named imports
`moment / locales`	Swap for date-fns / Temporal
`Duplicate packages`	resolve.dedupe / lockfile dedupe
`Unused CSS`	cssCodeSplit + purge via Tailwind

⚠ COMMON MISTAKES

- Using `dropConsole` and losing `console.error` too — it drops every `console.*` call; use `manualPureFunctions` for selective removal
- Importing `import _ from 'lodash'` — pulls the whole CJS library; import named functions from `lodash-es`
- Relying on `'esbuild'` minify after upgrading — it's no longer installed by default
- Judging size by uncompressed numbers — compare gzip/brotli sizes

✓ TIPS & BEST PRACTICES

- Add `"sideEffects": false` to your own library packages
- Check output with `npx vite-bundle-visualizer` for a one-off report
- Keep `debugger` removal on even if you keep console logs
- Try `build.chunkImportMap` (experimental) for better cache hits across deploys

QUICK SUMMARY

- `build.minify: 'oxc'` — default JS minifier
- `compress.dropConsole` — strip logs
- `rolldownOptions.treeshake` — tree-shaking control
- `visualizer()` — see what's in the bundle

Library Mode

BUILDING A PACKAGE

Library mode builds a reusable package instead of an app: no `index.html`, one or more JS entries, and output formats such as ES modules and UMD. The key job is marking framework dependencies as **external** so your library doesn't bundle its own copy of React or Vue. Pair the output with `exports` in `package.json` so consumers get the right file.

NOTES

- Default formats: `['es', 'umd']`, or `['es', 'cjs']` with multiple entries
- `name` is required for `umd` / `iife` (global variable)
- CSS is emitted as one file; name it with `build.lib.cssFileName`
- Types: generate `.d.ts` with `tsc` or `vite-plugin-dts`

VITE.CONFIG.TS

```ts
import { resolve } from 'node:path'
import { defineConfig } from 'vite'

export default defineConfig({
  build: {
    lib: {
      entry: {
        index: resolve(import.meta.dirname, 'src/index.ts'),
        utils: resolve(import.meta.dirname, 'src/utils.ts'),
      },
      formats: ['es', 'cjs'],
      fileName: (format, name) =>
        `${name}.${format === 'es' ? 'js' : 'cjs'}`,
      cssFileName: 'style',
    },
    rolldownOptions: {
      external: ['react', 'react-dom', 'react/jsx-runtime'],
    },
  },
})
```

PACKAGE.JSON

```json
{
  "name": "@acme/ui",
  "type": "module",
  "files": ["dist"],
  "exports": {
    ".": {
      "types": "./dist/index.d.ts",
      "import": "./dist/index.js",
      "require": "./dist/index.cjs"
    },
    "./utils": "./dist/utils.js",
    "./style.css": "./dist/style.css"
  },
  "peerDependencies": { "react": ">=18" },
  "sideEffects": ["**/*.css"]
}
```

BUILD.LIB OPTIONS

OPTION	PURPOSE
`entry`	Path, array or { name: path }
`name`	Global name for umd / iife
`formats`	es · cjs · umd · iife
`fileName`	String or (format, entry) => name
`cssFileName`	Name of the emitted CSS file

WHAT TO EXTERNALIZE

DEPENDENCY TYPE	EXTERNAL?
`peerDependencies (react, vue)`	✔ always
`dependencies`	✔ usually
`Subpaths (react/jsx-runtime)`	✔ list them too
`Small internal helpers`	✖ bundle
`Node built-ins (node:fs)`	✔ for Node libs

⚠ COMMON MISTAKES

- Leaving React un-externalized — consumers end up with two Reacts and broken hooks
- Externalizing `react` but not `react/jsx-runtime` — the runtime still gets bundled
- Forgetting `name` with a `umd` format — the build errors out
- Publishing without `"files": ["dist"]` — source and config ship too

✓ TIPS & BEST PRACTICES

- Use a regex external like `/^react(\/.*)?$/` to catch all subpaths
- Generate types with `vite-plugin-dts` or a separate `tsc --emitDeclarationOnly`
- Check the package with `npx publint` and `arethetypeswrong` before publishing
- Mark CSS as a side effect so bundlers don't drop it

QUICK SUMMARY

- `build.lib.entry` — library entry points
- `formats` — es / cjs / umd / iife
- `rolldownOptions.external` — keep peers out
- `package.json` `exports` — map entries to files

Preview & Static Deployment

SHIPPING DIST/

A Vite app builds to plain static files, so any static host works. `vite preview` serves `dist/` locally to check the build, but it's not a production server. The two settings that matter for deployment are **base** (when the site lives under a subpath) and an **SPA fallback** so client-side routes return `index.html` instead of 404.

NOTES

- Preview default port: `4173`
- `preview.*` options mirror `server.*` (host, port, proxy, headers)
- Hashed files in `assets/` can be cached forever
- `index.html` should not be cached long

PREVIEW AND A GITHUB PAGES DEPLOY

```
vite build && vite preview --port 8080 --host

# vite.config.ts for https://user.github.io/my-repo/
#   base: '/my-repo/',
#   preview: { port: 8080, strictPort: true },

# .github/workflows/deploy.yml (key steps)
#   - run: npm ci && npm run build
#   - uses: actions/upload-pages-artifact@v3
#     with: { path: dist }
#   - uses: actions/deploy-pages@v4

# SPA fallback examples
echo '/*  /index.html  200' > public/_redirects        # Netlify
# vercel.json: { "rewrites": [{ "source": "/(.*)", "destination": "/" }] }
```

CODE EXPLAINED

- → `vite preview` — local check of the real build
- → `base: '/my-repo/'` — project pages live under the repo name
- → `upload-pages-artifact` — publish `dist/` to Pages
- → `_redirects` in `public/` — copied to `dist/` for Netlify
- → `rewrites` — Vercel SPA fallback

STATIC HOSTS

HOST	BUILD CMD / OUTPUT	SPA FALLBACK
Netlify	npm run build / dist	_redirects or netlify.toml
Vercel	auto-detected / dist	vercel.json rewrites
Cloudflare Pages	npm run build / dist	Automatic for SPAs
GitHub Pages	Actions workflow / dist	Copy index.html → 404.html
nginx	copy dist/	try_files $uri /index.html;
Firebase Hosting	public: dist	rewrites in firebase.json

CACHE HEADERS

PATH	HEADER
/assets/*	max-age=31536000, immutable
/index.html	no-cache
/sw.js	no-cache

⚠ COMMON MISTAKES

- Running `vite preview` on a public server — it's not hardened for production traffic
- Deploying to `user.github.io/repo` with `base: '/'` — assets load from the wrong path
- Refreshing a deep link like `/settings` and getting 404 — the host needs an SPA fallback
- Caching `index.html` for a year — users never see new deployments

✓ TIPS & BEST PRACTICES

- Use `base: './'` when the same build is served from unknown subpaths
- Add `vite preview` to a CI smoke test with Playwright
- Keep old hashed assets for a while after deploying (open tabs still request them)
- Set `preview.headers` to test CSP and COOP before deploying

QUICK SUMMARY

- `vite preview` — local check only
- `base` — required for subpath hosting
- SPA fallback — route everything to `index.html`
- `assets/*` immutable, `index.html` no-cache

Browser Support & Legacy Builds

'baseline-widely-available' · build.target · @vitejs/plugin-legacy · modulePreload polyfill · no ES5 output

WHICH BROWSERS

Vite outputs modern JavaScript for browsers that support native ES modules, dynamic import and `import.meta`. The default `build.target` is **`'baseline-widely-available'`**: features available across major browsers for at least 30 months. Vite 8 raised that floor. For older browsers, `@vitejs/plugin-legacy` adds a second, polyfilled build loaded with `nomodule`.

NOTES

- Vite only transforms **syntax**; it doesn't add polyfills
- Use `'esnext'` for the smallest output on modern-only apps
- Passing the same browser twice in `target` now errors
- Vite 8: plugin-legacy can't output ES5 or lower

TARGETS AND THE LEGACY PLUGIN

```
import legacy from '@vitejs/plugin-legacy'

export default defineConfig({
  build: {
    target: ['chrome111', 'edge111', 'firefox114', 'safari16.4'],
    // target: 'esnext',            // newest syntax only
    cssTarget: 'chrome111',
    modulePreload: { polyfill: true },
  },
  plugins: [
    legacy({
      targets: ['defaults', 'not dead'],   // browserslist query
      modernPolyfills: true,               // polyfill modern build too
    }),
  ],
})
```

CODE EXPLAINED

- → `target` array — explicit browser versions
- → `'esnext'` — no syntax lowering at all
- → `cssTarget` — separate floor for CSS
- → `legacy({ targets })` — extra `nomodule` bundle + polyfills
- → `modernPolyfills` — core-js polyfills for the modern chunks

DEFAULT TARGET: VITE 7 → 8

BROWSER	VITE 7	VITE 8
Chrome	107	111
Edge	107	111
Firefox	104	114
Safari	16.0	16.4

TARGET VALUES

VALUE	MEANING
`'baseline-widely-available'`	Default
`'esnext'`	No lowering
`'es2022'`	ES version floor
`['chrome100', ...]`	Exact browsers
`legacy({ targets })`	Old browsers, extra build

⚠ COMMON MISTAKES

- Expecting `target: 'es2017'` to add `Array.prototype.at` — targets lower syntax only; polyfills need plugin-legacy or core-js
- Adding plugin-legacy for an internal app on evergreen browsers — doubles build time for nothing
- Listing `chrome100` and `chrome111` together in `target` — Vite 8 throws an error
- Upgrading to Vite 8 without noticing the raised baseline (Safari 16.0–16.3 users)

✓ TIPS & BEST PRACTICES

- Check real visitor browsers in analytics before choosing a target
- Keep one `browserslist` in package.json and reuse it for legacy and CSS
- Use `'esnext'` for Electron and Tauri apps with a known engine
- Test the legacy build in a real old browser via BrowserStack or similar

QUICK SUMMARY

- `build.target` — JS syntax floor
- `'baseline-widely-available'` — Vite default
- `@vitejs/plugin-legacy` — `nomodule` fallback build
- Syntax lowering ≠ polyfills

Using Plugins

THE PLUGINS ARRAY

Plugins extend Vite with frameworks, file types and build steps. Add them to the `plugins` array; each entry is a plugin object, an array of plugins, or a falsy value (ignored, which makes conditionals easy). Vite's plugin interface extends Rollup's, so many Rollup plugins work as-is, and the order of execution is controlled with `enforce` and `apply`.

NOTES

- Find plugins at `registry.vite.dev` and `github.com/vitejs/awesome-vite`
- Rollup plugins using standard hooks usually work with Rolldown
- Plugins relying on Rollup internals may break in Vite 8
- `vite-plugin-inspect` shows each plugin's transforms

ADDING AND ORDERING PLUGINS

```ts
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import svgr from 'vite-plugin-svgr'
import yaml from '@rollup/plugin-yaml'
import Inspect from 'vite-plugin-inspect'

export default defineConfig(({ command }) => ({
  plugins: [
    react(),
    svgr(),                                    // './x.svg?react'
    { ...yaml(), enforce: 'pre' },             // run before core plugins
    { ...someRollupPlugin(), apply: 'build' }, // build only
    command === 'serve' && Inspect(),          // dev only; false is ignored
  ],
}))
```

CODE EXPLAINED

- → Call the factory — `react()`, not `react`
- → `enforce: 'pre'` — run before Vite core plugins
- → `apply: 'build'` / `'serve'` — limit to one command
- → Falsy entries are skipped
- → Rollup plugins go in the same array

PLUGIN ORDER

STEP	RUNS
1	Alias
2	enforce: 'pre' plugins
3	Vite core plugins
4	Plugins with no enforce
5	Vite build plugins
6	enforce: 'post' plugins
7	Vite post-build (minify, manifest)

POPULAR PLUGINS

PACKAGE	ADDS
`vite-plugin-svgr`	SVG as React components
`vite-plugin-pwa`	Service worker + manifest
`vite-plugin-checker`	TS / ESLint in overlay
`vite-plugin-dts`	Library .d.ts files
`unplugin-auto-import`	Auto-import APIs
`vite-plugin-inspect`	Debug transforms

⚠ COMMON MISTAKES

- Passing `react` instead of `react()` — the factory must be called
- Adding a Rollup plugin that should only run on the final bundle without `apply: 'build'` — it slows or breaks dev
- Installing an unmaintained plugin that patches Rollup internals — check Vite 8 compatibility first
- Registering the same framework plugin twice through a meta-framework and your own config

✓ TIPS & BEST PRACTICES

- Use `vite-plugin-inspect` at `/__inspect/` to see what each plugin changed
- Wrap Rollup plugins with `{ ...plugin(), enforce, apply }` to tune them
- Prefer official `@vitejs/plugin-*` packages for frameworks
- Keep plugin count small; each transform hook costs time on every file

QUICK SUMMARY

- `plugins: [react()]` — register plugins
- `enforce` — pre / normal / post ordering
- `apply` — serve or build only
- Rollup plugins — mostly compatible

Framework Plugins

ONE PLUGIN PER FRAMEWORK

Each UI framework needs one plugin that compiles its components and wires up HMR. **`@vitejs/plugin-react` v6** uses Oxc for JSX and React Refresh and no longer depends on Babel; React Compiler is opt-in through `@rolldown/plugin-babel`. Vue, Svelte, Solid and Preact have their own official or framework-maintained plugins.

NOTES

- plugin-react v5 still works with Vite 8
- `@vitejs/plugin-react-swc` is an SWC-based alternative
- Vue JSX needs `@vitejs/plugin-vue-jsx` as well
- SvelteKit / Nuxt add these plugins for you

FRAMEWORK SETUPS

```
// React (+ optional React Compiler)
import react, { reactCompilerPreset } from '@vitejs/plugin-react'
import babel from '@rolldown/plugin-babel'
plugins: [react(), babel({ presets: [reactCompilerPreset()] })]

// Vue 3 (+ JSX)
import vue from '@vitejs/plugin-vue'
import vueJsx from '@vitejs/plugin-vue-jsx'
plugins: [
  vue({ template: { compilerOptions: {
    isCustomElement: (tag) => tag.startsWith('ion-'),
  } } }),
  vueJsx(),
]

// Svelte 5
import { svelte } from '@sveltejs/vite-plugin-svelte'
plugins: [svelte()]

// Solid / Preact
import solid from 'vite-plugin-solid'        // plugins: [solid()]
import preact from '@preact/preset-vite'     // plugins: [preact()]
```

CODE EXPLAINED

- → `react()` — JSX, Fast Refresh, automatic runtime
- → `reactCompilerPreset()` — React Compiler via Babel
- → `vue()` — SFC compile; `isCustomElement` for web components
- → `svelte()` — reads `svelte.config.js`
- → `preact()` — aliases React imports to Preact

OFFICIAL / RECOMMENDED PLUGINS

FRAMEWORK	PACKAGE
React	@vitejs/plugin-react
Vue 3	@vitejs/plugin-vue
Svelte	@sveltejs/vite-plugin-svelte
Solid	vite-plugin-solid
Preact	@preact/preset-vite

REACT() OPTIONS (V6)

OPTION	PURPOSE
`include / exclude`	Files to transform
`jsxImportSource`	e.g. '@emotion/react'
`jsxRuntime`	'automatic' (default) · 'classic'
`reactRefreshHost`	Module federation HMR
`compiler`	Experimental Rust React Compiler

⚠ COMMON MISTAKES

- Passing `babel: { plugins: [...] }` to plugin-react v6 — Babel was removed; use `@rolldown/plugin-babel`
- Using both `plugin-react` and `plugin-react-swc` — they conflict; pick one
- Forgetting `vueJsx()` and getting JSX syntax errors in `.tsx` Vue files
- Adding `svelte()` to a SvelteKit config — `sveltekit()` already includes it

✓ TIPS & BEST PRACTICES

- Start with plain `react()`; add React Compiler only after measuring
- Use `compilationMode: 'annotation'` to try React Compiler on a few components
- Set Emotion with `react({ jsxImportSource: '@emotion/react' })`
- Stay on plugin-react v5 if you depend on Babel plugins and can't migrate yet

QUICK SUMMARY

- `react()` — Oxc-based in v6, no Babel
- `reactCompilerPreset()` — opt-in React Compiler
- `vue()` / `svelte()` / `solid()` — per framework
- One framework plugin per project

Writing Plugins: Core Hooks

PLUGIN ANATOMY

A Vite plugin is an object with a `name` and **hooks**. Universal hooks (`resolveId`, `load`, `transform`) come from the Rollup/Rolldown API and run in both dev and build. Vite-specific hooks (`config`, `configResolved`, `configureServer`…) adjust Vite itself. Return the plugin from a factory function so users can pass options.

NOTES

- Prefix virtual module ids with `\0` after resolving
- Use `virtual:` in the public import name by convention
- Hook **filters** skip the JS call for files that don't match
- Name community plugins `vite-plugin-*`

A VIRTUAL MODULE + TRANSFORM PLUGIN

```ts
import type { Plugin } from 'vite'

export default function buildInfo(): Plugin {
  const id = 'virtual:build-info'
  const resolved = '\0' + id
  let mode = ''
  return {
    name: 'build-info',
    configResolved(config) { mode = config.mode },
    resolveId(source) { if (source === id) return resolved },
    load(loadId) {
      if (loadId === resolved) return `export const mode = '${mode}'`
    },
    transform: {
      filter: { id: /\.md$/ },                // only Markdown files
      handler(code) { return `export default ${JSON.stringify(code)}` },
    },
  }
}
// app: import { mode } from 'virtual:build-info'
```

CODE EXPLAINED

- → Factory function returns the plugin object
- → `resolveId` — claim the virtual id
- → `load` — return source code for that id
- → `transform.filter` — hook only runs on matches
- → `configResolved` — read the final config

UNIVERSAL HOOKS

HOOK	PURPOSE
`options / buildStart`	Setup per build
`resolveId(source, importer)`	Map specifier → id
`load(id)`	Provide source for an id
`transform(code, id)`	Modify source
`buildEnd / closeBundle`	Cleanup
`generateBundle`	Edit output (build only)

VITE-ONLY HOOKS

HOOK	PURPOSE
`config(config, env)`	Return partial config
`configResolved(c)`	Store final config
`configureServer(s)`	Dev middleware (sheet 30)
`transformIndexHtml`	Edit HTML (sheet 30)
`hotUpdate`	Custom HMR (sheet 30)

⚠ COMMON MISTAKES

- Returning the virtual id without `\0` — other plugins (and sourcemaps) try to treat it as a real file
- Transforming every file without a filter or id check — slows the whole dev server
- Mutating `config` in `configResolved` — it's read-only by then; use `config()`
- Returning a string from `transform` without a sourcemap — debugging maps break; return `{ code, map }`

✓ TIPS & BEST PRACTICES

- Use hook `filter` objects (`id`, `code`) so Rolldown can skip non-matching files natively
- Add a `.d.ts` for your virtual module: `declare module 'virtual:build-info'`
- Test hooks with `vite-plugin-inspect` to see before/after output
- Use `this.environment` (sheet 30/32) instead of `ssr` flags in new plugins

QUICK SUMMARY

- `resolveId` + `load` — virtual modules
- `transform` — change source code
- `config` / `configResolved` — Vite settings
- `filter` — cheap file matching

Writing Plugins: Server & HTML Hooks

DEV-SERVER HOOKS

Some plugin hooks exist only in Vite. **`configureServer`** gives you the dev server to add middleware or watch files; **`transformIndexHtml`** edits or injects into HTML; **`hotUpdate`** customizes what happens when a file changes. Inside hooks, `this.environment` tells you whether you're working on the client, SSR or another environment.

NOTES

- `configureServer` can return a function to run **after** internal middleware
- `configurePreviewServer` is the same hook for `vite preview`
- `hotUpdate` replaces the older `handleHotUpdate`
- Use `apply: 'serve'` for dev-only plugins

MIDDLEWARE, HTML INJECTION, HMR

```ts
import type { Plugin } from 'vite'

export function devTools(): Plugin {
  return {
    name: 'dev-tools',
    apply: 'serve',
    configureServer(server) {
      server.middlewares.use('/__health', (req, res) => res.end('ok'))
      return () => server.middlewares.use(spaFallback)   // after internals
    },
    transformIndexHtml: () => [
      { tag: 'script', attrs: { src: '/banner.js' }, injectTo: 'body' },
    ],
    hotUpdate({ file, server }) {
      if (file.endsWith('.md') && this.environment.name === 'client') {
        server.ws.send({ type: 'custom', event: 'content-changed', data: { file } })
        return []                                          // skip default HMR
      }
    },
  }
}
```

CODE EXPLAINED

→ `server.middlewares` — Connect app; add routes

→ Returned function — post-middleware

→ `transformIndexHtml` — return tags or new HTML

→ `hotUpdate` — `[]` = handled; `server.ws.send` notifies clients

USEFUL SERVER PROPERTIES

PROPERTY	WHAT IT IS
`server.middlewares`	Connect app
`server.watcher`	File watcher
`server.ws`	HMR WebSocket (send / on)
`server.environments`	client, ssr, …

TRANSFORMINDEXHTML RETURN

RETURN VALUE	EFFECT
`string`	Replace HTML
`[{ tag, attrs, injectTo }]`	Inject tags
`{ html, tags }`	Both
`{ order: 'pre', handler }`	Run before other HTML transforms

⚠ COMMON MISTAKES

- Adding a catch-all middleware directly in `configureServer` — it runs before Vite's and swallows module requests; return a function instead
- Forgetting `apply: 'serve'` on dev-only plugins — `configureServer` code may still be imported and run at build
- Using `handleHotUpdate` in new plugins — prefer `hotUpdate`, which is environment-aware

✓ TIPS & BEST PRACTICES

- Debug your transforms with `vite-plugin-inspect` at `/__inspect/`
- Use `server.ws.send({ type: 'full-reload' })` to force a reload
- Use `injectTo: 'head-prepend'` for scripts that must run first
- Read `this.environment.config` instead of global config in environment-aware hooks

QUICK SUMMARY

- `configureServer` — dev middleware + watcher
- `transformIndexHtml` — edit / inject HTML
- `hotUpdate` — custom HMR handling
- `this.environment` — which environment is running

Server-Side Rendering

DIY SSR WITH VITE

Vite provides SSR **building blocks**, not a framework: your own Node server runs Vite in **middleware mode**, loads the server entry with `ssrLoadModule()` (with HMR), renders HTML and sends it. For production you build twice, once for the client and once with `--ssr` for the server. Most teams use a framework that wires this up (Nuxt, SvelteKit, React Router, Astro, Vike).

NOTES

- Typical files: `entry-client.ts` (hydrate) + `entry-server.ts` (render)
- `appType: 'custom'` turns off Vite's own HTML serving
- Call `vite.ssrFixStacktrace(e)` in catch blocks for real line numbers
- `import.meta.env.SSR` is `true` in server code

DEV SERVER (EXPRESS 5)

```
import fs from 'node:fs/promises'
import express from 'express'
import { createServer } from 'vite'
const app = express()
const vite = await createServer({
  server: { middlewareMode: true },
  appType: 'custom',
})
app.use(vite.middlewares)                    // assets, HMR, transforms
app.use('*all', async (req, res) => {
  const url = req.originalUrl
  let html = await fs.readFile('index.html', 'utf-8')
  html = await vite.transformIndexHtml(url, html)     // inject HMR client
  const { render } = await vite.ssrLoadModule('/src/entry-server.ts')
  html = html.replace('<!--app-html-->', await render(url))
  res.status(200).set({ 'Content-Type': 'text/html' }).end(html)
})
app.listen(5173)
```

CODE EXPLAINED

- → `middlewareMode` — your server owns the HTTP server
- → `vite.middlewares` — serves modules and assets
- → `transformIndexHtml` — applies Vite and plugin HTML transforms
- → `ssrLoadModule` — loads TS/JSX in Node, re-runs on change
- → `<!--app-html-->` — placeholder you replace with rendered markup

VITE BUILD FLAGS FOR SSR

FLAG	OUTPUT
`--outDir dist/client`	Client assets + index.html
`--ssr src/entry-server.ts`	Server module (add --outDir dist/server)
`--ssrManifest`	Module → chunk map for preloads
`--app`	All environments (sheet 32)

SSR.* OPTIONS

OPTION	PURPOSE
`ssr.external`	Deps to keep as require/import
`ssr.noExternal`	Deps to bundle (CSS imports, ESM-only)
`ssr.target`	'node' (default) or 'webworker'
`ssr.resolve.conditions`	Export conditions for SSR

⚠ COMMON MISTAKES

- Reading `dist/client/index.html` in dev or `index.html` in production — keep separate code paths for each
- Touching `window` or `document` at module top level in shared code — it crashes during SSR; guard with `import.meta.env.SSR`
- Leaving a UI library external when it imports CSS — Node can't load `.css`; add it to `ssr.noExternal`

✓ TIPS & BEST PRACTICES

- Start from `npm create vite-extra@latest` SSR templates if you build your own
- Use the SSR manifest to emit `<link rel="modulepreload">` for the rendered route
- Prefer a meta-framework unless you need full control

QUICK SUMMARY

- `middlewareMode` + `appType: 'custom'` — DIY server
- `ssrLoadModule()` — run server entry in dev
- `vite build --ssr` — server bundle
- `ssr.noExternal` — bundle problem deps

Environment API

MANY RUNTIMES, ONE SERVER

The **Environment API** generalizes Vite's old client/SSR split into any number of named environments: browser client, Node SSR, edge workers, React Server Components and so on. Each environment gets its own module graph, resolve conditions and build output. It's aimed mainly at framework and runtime authors; app developers usually get it through their framework.

NOTES

- Introduced in Vite 6; still marked as not yet stable in Vite 8
- Top-level options act as defaults for every environment
- `vite build --app` builds all environments
- Plugins read `this.environment` instead of an `ssr` flag

CONFIG

```ts
export default defineConfig({
  build: { sourcemap: true },        // default for all
  environments: {
    client: { build: { outDir: 'dist/client' } },
    ssr: { build: { outDir: 'dist/server' } },
    edge: {
      resolve: { noExternal: true, conditions: ['workerd'] },
      build: {
        outDir: 'dist/edge',
        rolldownOptions: { input: 'src/edge.ts' },
      },
    },
  },
  builder: {
    buildApp: (b) => Promise.all(
      Object.values(b.environments).map((e) => b.build(e))),
  },
})
```

USING ENVIRONMENTS

```ts
import { createServer, isRunnableDevEnvironment } from 'vite'

const server = await createServer()
const ssr = server.environments.ssr

if (isRunnableDevEnvironment(ssr)) {
  const { render } = await ssr.runner.import('/src/entry-server.ts')
  console.log(await render('/'))
}

// In a plugin
const plugin = {
  name: 'env-aware',
  applyToEnvironment: (env) => env.name !== 'client',
  transform(code, id) {
    const server = this.environment.config.consumer === 'server'
  },
}
```

KEY TERMS

TERM	MEANING
`environment`	Named runtime with its own graph
`consumer`	'client' or 'server'
`DevEnvironment`	Dev-time instance per env
`RunnableDevEnvironment`	Can run modules in-process
`ModuleRunner`	Executes modules (vite/module-runner)
`builder`	Builds every environment

PLUGIN HOOKS FOR ENVIRONMENTS

API	USE
`this.environment`	Current environment in hooks
`applyToEnvironment(env)`	Enable plugin per env
`configEnvironment(name, cfg)`	Adjust one env's config
`hotUpdate`	Per-environment HMR
`sharedDuringBuild`	Share one instance across envs

⚠ COMMON MISTAKES

- Treating the API as stable for app code — expect changes between minors until it's marked stable
- Checking `options.ssr` in new plugin hooks — use `this.environment.name` or `.config.consumer`
- Assuming every environment can run in Node — edge or workerd environments need their own runner
- Building with plain `vite build` and only getting the client — use `vite build --app` or `builder.buildApp`

✓ TIPS & BEST PRACTICES

- Use `isRunnableDevEnvironment()` before calling `.runner.import()`
- Put shared settings at the top level; override only what differs per environment
- Read the framework's docs first; most expose a higher-level API
- Test plugins against both `client` and `ssr` environments

QUICK SUMMARY

- `environments: { client, ssr, … }` — per-runtime config
- `server.environments.ssr.runner.import()` — run code in dev
- `vite build --app` — build every environment
- `this.environment` — environment-aware plugins

JavaScript API

VITE FROM NODE

Everything the CLI does is available from Node through the `vite` package. Use it to embed a dev server in your own tool, script builds, run several builds in sequence, or read the resolved config. Each function takes an **inline config** that's merged with the project's config file unless you pass `configFile: false`.

NOTES

- All functions are async; Vite is ESM-only
- Inline config has extra keys: `configFile`, `envFile`
- `build()` returns the output, or a watcher with `build.watch`
- `transformWithEsbuild` → `transformWithOxc` in Vite 8

SCRIPTING VITE

```
import { createServer, build, preview, resolveConfig,
         mergeConfig, loadEnv, transformWithOxc } from 'vite'

const server = await createServer({
  configFile: false, root: import.meta.dirname, server: { port: 1337 },
})
await server.listen()
server.printUrls()
server.bindCLIShortcuts({ print: true })

await build({ root: './packages/app', build: { outDir: 'dist' } })
await build({ build: { ssr: 'src/entry-server.ts', outDir: 'dist/server' } })
const pv = await preview({ preview: { port: 8080, open: true } })

const config = await resolveConfig({}, 'build')   // command
const env = loadEnv('production', process.cwd(), '')
const { code } = await transformWithOxc('let n: number = 1', 'n.ts')
```

CODE EXPLAINED

- → `createServer()` → `ViteDevServer`; call `listen()`
- → `configFile: false` — ignore vite.config.*
- → `build()` — sequential calls for client + server
- → `resolveConfig(inline, command)` — final config object
- → `transformWithOxc` — one-off TS/JSX transform

EXPORTED FUNCTIONS

FUNCTION	RETURNS / USE
`createServer(cfg)`	ViteDevServer
`build(cfg)`	Output or watcher
`createBuilder(cfg)`	Builder for all environments
`preview(cfg)`	PreviewServer
`resolveConfig(cfg, cmd)`	ResolvedConfig
`mergeConfig(a, b)`	Deep-merged config
`loadEnv(mode, dir, prefix)`	Env object
`loadConfigFromFile(env)`	Config from disk

VITEDEVSERVER METHODS

METHOD	USE
`listen(port?)`	Start listening
`printUrls()`	Log Local / Network URLs
`transformRequest(url)`	Transform a module
`ssrLoadModule(url)`	Load module for SSR
`restart()`	Restart with new config
`close()`	Stop the server

⚠ COMMON MISTAKES

- Calling `createServer()` and expecting it to listen — you must `await server.listen()`
- Running `build()` twice in parallel on the same outDir — builds overwrite each other; await them in order
- Forgetting `configFile: false` when you mean to ignore the project's config
- Using `__dirname` in an ESM script — use `import.meta.dirname`

✓ TIPS & BEST PRACTICES

- Use `searchForWorkspaceRoot(process.cwd())` to set `server.fs.allow` in monorepos
- Close servers in tests with `await server.close()` to avoid hanging processes
- Use `normalizePath()` for Windows-safe path comparisons in tools
- Prefer the CLI unless you need programmatic control

QUICK SUMMARY

- `createServer()` + `listen()` — dev server
- `build()` / `preview()` — production steps
- `resolveConfig()` — inspect final config
- `transformWithOxc()` — standalone transform

Performance & Bundled Dev Mode

SPEEDING UP DEV

Vite's dev server is fast by default, but very large apps can hit limits: thousands of module requests on first load, slow plugins, and deep barrel-file imports. Fix the common causes first (warm-up, direct imports, fewer heavy plugins). Vite 8.1 also adds an experimental **Bundled Dev Mode** that serves bundled output in development for apps with huge module graphs.

NOTES

- Bundled Dev Mode was previously called Full Bundle Mode
- It may not work with every third-party plugin yet
- `--profile` + `p` key records a CPU profile
- Vite Devtools (`devtools: true`) shows transform timing

CONFIG AND TOOLS

```
export default defineConfig({
  server: {
    warmup: {
      clientFiles: ['./src/main.tsx', './src/components/**/*.tsx'],
      ssrFiles: ['./src/entry-server.ts'],
    },
  },
  experimental: { bundledDev: true },  // Vite 8.1+, or: vite --experimental-bundle
  devtools: true,                      // Vite Devtools
})

// Avoid barrel files in hot paths
import { Button } from '@/components'              // loads every component
import { Button } from '@/components/Button.tsx'   // loads one file

// Profiling (shell)
// vite --profile        → press p+enter, open .cpuprofile in DevTools
// vite --debug plugin-transform   → slow plugin transforms
```

CODE EXPLAINED

- → `warmup.clientFiles` — transform likely files before requests
- → `bundledDev` — serve bundled modules in dev (experimental)
- → `devtools` — inspect modules and plugin timings
- → Direct imports — skip loading every re-exported file
- → `--profile` / `--debug` — find the slow part

COMMON SLOWDOWNS

SYMPTOM	FIX
Slow first page load	server.warmup, fewer modules
Thousands of requests	Avoid barrels; try bundledDev
Slow transforms	Find plugin with --debug, add filters
Re-optimize reloads	Add deps to optimizeDeps.include
Slow resolve	Explicit extensions in imports
Slow build	reportCompressedSize: false

PLUGIN HYGIENE

PRACTICE	WHY
Hook filters	Skip JS calls for non-matching files
apply: 'build'	Keep build-only work out of dev
Cache expensive work	Same file transforms often
Avoid full AST parses	Check code.includes() first
Drop unused plugins	Each hook adds per-file cost

⚠ COMMON MISTAKES

- Enabling `bundledDev` and assuming plugins behave the same — test HMR and custom plugins carefully
- Warming up every file with `**/*` — wastes startup time on files you never open
- Re-exporting an entire folder through `index.ts` and importing from it everywhere
- Blaming Vite before checking browser extensions — some slow down every module request

✓ TIPS & BEST PRACTICES

- Open DevTools' Network panel with cache disabled off; Vite relies on browser caching
- Run `vite --profile` and load the page once, then inspect the profile in Chrome or speedscope
- Use `vite-plugin-inspect` to compare per-plugin transform time
- Keep the dev server on fast storage (not network drives or slow WSL mounts)

QUICK SUMMARY

- `server.warmup` — pre-transform hot files
- `experimental.bundledDev` — bundled dev (8.1)
- Direct imports — avoid barrel files
- `--profile` / `devtools` — measure first

Testing with Vitest

VITE-NATIVE TESTING

Vitest is the test runner built on Vite. It reuses your `vite.config` (plugins, aliases, transforms), so tests see code exactly as the app does, and it has a Jest-compatible API (`describe`, `it`, `expect`, `vi`). Add a `test` block to your Vite config or a separate `vitest.config.ts`.

NOTES

- `vitest` watches by default; `vitest run` runs once (CI)
- Install `jsdom` or `happy-dom` for DOM tests
- Browser Mode runs tests in a real browser via Playwright
- Use a Vitest release that supports your Vite major

VITE.CONFIG.TS

```ts
/// <reference types="vitest/config" />
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'

export default defineConfig({
  plugins: [react()],
  test: {
    environment: 'jsdom',
    globals: true,          // no test imports
    setupFiles: ['./src/test/setup.ts'],
    include: ['src/**/*.{test,spec}.{ts,tsx}'],
    coverage: {
      provider: 'v8', reporter: ['text', 'html'],
    },
  },
})
```

PRICE.TEST.TS

```ts
import { describe, it, expect, vi } from 'vitest'
import { total } from './price'
import { getRate } from './api'

vi.mock('./api', () => ({ getRate: vi.fn() }))

describe('total', () => {
  it('adds tax', async () => {
    vi.mocked(getRate).mockResolvedValue(0.1)
    expect(await total(100)).toBe(110)
  })
})
```

COMMANDS

COMMAND	DOES
`vitest`	Watch mode
`vitest run`	Single run (CI)
`vitest run --coverage`	With coverage
`vitest -t "adds tax"`	Filter by test name
`vitest related src/a.ts`	Tests touching a file
`vitest --ui`	Browser UI (@vitest/ui)

CORE API

API	USE
`expect(x).toBe / toEqual`	Assertions
`vi.fn() / vi.spyOn()`	Mocks and spies
`vi.mock(path, factory)`	Mock a module (hoisted)
`vi.useFakeTimers()`	Control time
`beforeEach / afterEach`	Setup / teardown
`import.meta.vitest`	In-source tests

⚠ COMMON MISTAKES

- Using `globals: true` without adding `vitest/globals` to tsconfig `types` — tests run but the editor complains
- Expecting `vi.mock` to run where it's written — it's hoisted to the top of the file
- Forgetting to install `jsdom` after setting `environment: 'jsdom'`
- Keeping Jest's `moduleNameMapper` — Vitest uses your Vite `resolve.alias` instead

✓ TIPS & BEST PRACTICES

- Use `mergeConfig(viteConfig, defineConfig({ test }))` in a separate `vitest.config.ts` if you prefer
- Add `@testing-library/jest-dom` matchers in the setup file
- Use workspaces/projects for monorepos with mixed environments
- Run `vitest run --coverage` in CI with thresholds

QUICK SUMMARY

- `test: {}` in vite.config — Vitest settings
- `vitest` / `vitest run` — watch vs CI
- `vi.mock` / `vi.fn` — mocking
- `environment: 'jsdom'` — DOM tests

Debugging & Troubleshooting

FINDING THE PROBLEM

Most Vite problems fall into a few groups: dependency optimization, module resolution, CommonJS interop, host and file-system security checks, and stale caches. Turn on debug logging to see what Vite is doing, check the browser console and network tab, and clear the dependency cache before digging deeper. Vite 8 also changed how CommonJS `default` imports resolve, which surfaces in upgraded projects.

NOTES

- `vite --force` clears the dep cache for one run
- `server.forwardConsole` shows browser errors in the terminal
- `build()` errors are now a `BundleError` with an `errors` array
- `legacy.inconsistentCjsInterop: true` restores old CJS behavior temporarily

DEBUG COMMANDS

```
vite --debug                    # all debug logs
vite --debug hmr                # why HMR did a full reload
vite --debug deps               # dependency optimizer
DEBUG="vite:resolve-details" vite   # resolution decisions
vite --force                    # rebuild node_modules/.vite
rm -rf node_modules/.vite       # same, permanently
vite build --debug --logLevel info
npx vite --version && node -v   # check versions first

# vite.config.ts
#   server: { forwardConsole: true, hmr: { overlay: true } }
#   logLevel: 'info', clearScreen: false
```

CODE EXPLAINED

- → `--debug <ns>` — filter logs to one area
- → `DEBUG=vite:*` — same via environment variable
- → `--force` — first thing to try after dep changes
- → `--logLevel info` — more detail in builds
- → Version check — many issues are version mismatches

COMMON ERRORS

MESSAGE	FIX
`Failed to resolve import "x"`	Install the dep, fix the path or alias
`Blocked request. This host … is not allowed`	Add it to server.allowedHosts
`… outside of Vite serving allow list`	Add folder to server.fs.allow
`504 (Outdated Optimize Dep)`	Reload; vite --force; optimizeDeps.include
`does not provide an export named 'default'`	CJS interop: include dep; check import style
`require is not defined`	CJS in browser code: include in optimizeDeps
`Pre-transform error: … JSX`	Rename file to .jsx/.tsx or add plugin
`Port 5173 is in use`	Another server running; --port or --strictPort

⚠ COMMON MISTAKES

- Deleting `node_modules` and the lockfile for every problem — try `vite --force` first
- Suppressing the error overlay instead of fixing the error behind it
- Mixing `import x from 'cjs-pkg'` and `import * as x` after upgrading to Vite 8 without checking the new interop rules
- Debugging a production-only bug in dev — reproduce with `vite build && vite preview`

✓ TIPS & BEST PRACTICES

- Reproduce on StackBlitz (`vite.new`) before filing an issue
- Search the Vite GitHub issues with the exact error text
- Use `vite-plugin-inspect` to see what a plugin did to the failing file
- Turn on `server.forwardConsole` when testing on phones or in agents

QUICK SUMMARY

- `--debug [ns]` — targeted logs
- `vite --force` — clear dep cache
- `server.fs.allow` / `allowedHosts` — security errors
- `legacy.inconsistentCjsInterop` — temporary CJS escape hatch

Migrating to Vite 8

WHAT CHANGED

Vite 8 swaps esbuild and Rollup for **Rolldown** and **Oxc**. A compatibility layer converts most old options automatically, so many projects upgrade with no config changes, but you'll see deprecation warnings until you rename them. The breaking changes are a higher browser baseline, new default minifiers, stricter CommonJS interop and some removed Rollup-only options and hooks.

NOTES

- Node requirement unchanged: 20.19+ / 22.12+
- Install grows by about 15 MB (Lightning CSS, Rolldown)
- `@vitejs/plugin-react` v5 still works; v6 drops Babel
- Try the upgrade on a branch and run a full build + preview

BEFORE (VITE 7)

```
export default defineConfig({
  esbuild: {
    jsxImportSource: 'preact',
    drop: ['console', 'debugger'],
  },
  optimizeDeps: {
    esbuildOptions: { keepNames: true },
  },
  build: {
    rollupOptions: {
      output: {
        manualChunks: { vendor: ['react', 'react-dom'] },
      },
    },
  },
})
```

AFTER (VITE 8)

```
export default defineConfig({
  oxc: { jsx: { importSource: 'preact' } },
  optimizeDeps: {
    rolldownOptions: { output: { keepNames: true } },
  },
  build: {
    rolldownOptions: {
      output: {
        minify: { compress: { dropConsole: true, dropDebugger: true } },
        codeSplitting: {
          groups: [{ name: 'vendor', test: /node_modules[\\/]react/ }],
        },
      },
    },
  },
})
```

RENAMED OPTIONS

VITE 7	VITE 8
`build.rollupOptions`	build.rolldownOptions
`worker.rollupOptions`	worker.rolldownOptions
`optimizeDeps.esbuildOptions`	optimizeDeps.rolldownOptions
`esbuild`	oxc
`esbuild.drop`	output.minify.compress.drop*
`transformWithEsbuild`	transformWithOxc
`output.manualChunks`	output.codeSplitting

CHANGED DEFAULTS & REMOVALS

CHANGE	OPT OUT / FIX
`JS minify: Oxc`	minify: 'esbuild' (deprecated)
`CSS minify: Lightning CSS`	cssMinify: 'esbuild'
`Target: Safari 16.4 etc.`	Set build.target
`CJS default import rules`	legacy.inconsistentCjsInterop
`build.commonjsOptions`	No-op; remove it
`format 'system' / 'amd'`	Not supported
`hot.accept(url)`	Pass an id instead

⚠ COMMON MISTAKES

- Upgrading plugins and Vite in one step — upgrade Vite first, confirm it builds, then plugins
- Ignoring deprecation warnings because the build passes — the auto-conversion will go away in a later major
- Assuming `esbuild` is still installed — add it as a devDependency if you keep `minify: 'esbuild'`
- Missing the CJS default-import change — test pages that use older CommonJS packages

✓ TIPS & BEST PRACTICES

- Run `npx vite build` and read every deprecation line; each one names the new option
- Compare bundle sizes before and after (CSS may grow slightly with Lightning CSS)
- Check plugins that use `shouldTransformCachedModule`, `resolveImportMeta`, `renderDynamicImport` or `resolveFileUrl`; they're unsupported
- Read the official "Migration from v7" guide for the full list

QUICK SUMMARY

- `rollupOptions` → `rolldownOptions`
- `esbuild` → `oxc`
- `manualChunks` → `codeSplitting.groups`
- Oxc + Lightning CSS — new default minifiers

Quick Reference: CLI & Config

CLI flags · config skeleton · server options · build options · resolve / css options · defaults

COMMANDS

COMMAND	DOES
`vite`	Dev server :5173
`vite build`	Build → dist/
`vite preview`	Serve dist/ :4173
`vite build --watch`	Rebuild on change
`vite build --ssr <entry>`	Server bundle
`vite build --app`	All environments
`npm create vite@latest`	New project

COMMON FLAGS

FLAG	DOES
`--host`	Listen on all interfaces
`--port <n>`	Port
`--strictPort`	Fail if port busy
`--open`	Open browser
`--force`	Re-run dep optimizer
`-m, --mode <m>`	Mode + .env file
`-c, --config <f>`	Config file

BUILD FLAGS

FLAG	DOES
`--outDir <d>`	Output folder
`--sourcemap`	true · inline · hidden
`--minify <m>`	oxc · terser · false
`--emptyOutDir`	Clear outDir
`--manifest`	Write manifest
`--base <p>`	Public base path
`--profile`	CPU profile

TOP-LEVEL OPTIONS

OPTION	DEFAULT
`root`	process.cwd()
`base`	'/'
`publicDir`	'public'
`define`	{}
`envPrefix`	'VITE_'
`appType`	'spa'
`plugins`	[]

SERVER.*

OPTION	DEFAULT
`host`	'localhost'
`port`	5173
`proxy`	{ '/api': target }
`allowedHosts`	[]
`fs.allow`	workspace root
`warmup`	{ clientFiles }
`forwardConsole`	auto

BUILD.*

OPTION	DEFAULT
`outDir`	'dist'
`target`	baseline-widely-available
`minify / cssMinify`	'oxc' / 'lightningcss'
`sourcemap`	false
`rolldownOptions`	{ input, output }
`lib`	{ entry, formats }
`assetsInlineLimit`	4096

RESOLVE.* / CSS.*

OPTION	USE
`resolve.alias`	{ '@': path }
`resolve.tsconfigPaths`	true
`resolve.dedupe`	['react']
`css.modules`	Naming rules
`css.preprocessorOptions`	{ scss }
`css.transformer`	'lightningcss'

OTHER BLOCKS

OPTION	USE
`optimizeDeps.include`	Force pre-bundle
`optimizeDeps.exclude`	Serve as source
`preview.port`	4173
`ssr.noExternal`	Bundle for SSR
`worker.format`	'iife' · 'es'
`environments`	{ client, ssr, … }

CONFIG SKELETON

KEY	VALUE
`import`	{ defineConfig } from 'vite'
`export default`	defineConfig(({ command, mode }) => ({
`plugins`	[react()],
`resolve`	{ alias: { '@': src } },
`server`	{ port: 3000, proxy },
`build`	{ outDir: 'dist' },
`}))`	

Quick Reference: Imports, Env & Plugin API

IMPORT SUFFIXES

IMPORT	RETURNS
`'./a.png'`	URL
`'./a.svg?url'`	URL, never inlined
`'./a.txt?raw'`	String contents
`'./a.png?inline'`	data: URL
`'./a.css?inline'`	CSS string
`'./w.ts?worker'`	Worker class
`'./m.wasm?init'`	init() → Instance

IMPORT.META.GLOB

FORM	RESULT
`glob('./x/*.ts')`	{ path: () => import }
`{ eager: true }`	{ path: module }
`{ import: 'default' }`	One export only
`{ query: '?raw' }`	Suffix on each
`['./a/*', '!**/b.ts']`	Multi + exclude
`{ caseSensitive: false }`	Ignore case (8.1)

IMPORT.META.ENV

KEY	VALUE
`MODE`	Current mode
`DEV / PROD`	Booleans
`SSR`	Server-side?
`BASE_URL`	base option
`VITE_*`	From .env files
`loadEnv(m, dir, '')`	In vite.config

IMPORT.META.HOT

API	USE
`accept(cb?)`	Self-accept
`accept(dep, cb)`	Accept a dep
`dispose(cb)`	Cleanup
`data`	Persisted state
`invalidate()`	Force propagate
`on(event, cb)`	Listen

.ENV LOAD ORDER

FILE	WHEN
`.env`	Always
`.env.local`	Always, git-ignored
`.env.[mode]`	Matching mode
`.env.[mode].local`	Mode, highest priority
`Shell env`	Beats all files

DYNAMIC IMPORTS

PATTERN	NOTES
`import('./x.ts')`	Own chunk
`` `./dir/${v}.json` ``	Globbed, 1 level
`/* @vite-ignore */`	Skip analysis
`new URL(p, import.meta.url)`	Asset URL
`vite:preloadError`	Missing chunk event

PLUGIN HOOKS: UNIVERSAL

HOOK	PURPOSE
`resolveId`	Specifier → id
`load`	Source for id
`transform`	Change code
`buildStart / buildEnd`	Lifecycle
`generateBundle`	Edit output
`{ filter, handler }`	Filtered hook

PLUGIN HOOKS: VITE

HOOK	PURPOSE
`config`	Modify config
`configResolved`	Read final config
`configureServer`	Dev middleware
`transformIndexHtml`	Edit HTML
`hotUpdate`	Custom HMR
`applyToEnvironment`	Per environment

JAVASCRIPT API

FUNCTION	RETURNS
`createServer()`	ViteDevServer
`build()`	Output / watcher
`preview()`	PreviewServer
`resolveConfig()`	ResolvedConfig
`mergeConfig()`	Merged config
`transformWithOxc()`	{ code, map }